

UNIVERSIDAD DE LAS FUERZAS ARMADAS ESPE
MATRIZ SANGOLQUÍ
DEPARTAMENTO DE CIENCIAS DE LA COMPUTACIÓN
CARRERA DE INGENIERÍA EN SOFTWARE

Título:	Proyecto Integrador del Primer Parcial Backend V1: autenticación JWT/RBAC y detección de esteganografía
Autores:	• Ariel Morillo • Matias Lugmaña • Alexander Villacrés
Carrera:	Ingeniería en Software
Asignatura:	Desarrollo de Software Seguro
Docente:	Ing. Angel Cudco.
Fecha de entrega:	04/11/2026
Revisión documental:	05/11/2026

Índice general

Índice general 2

1. Presentación y alcance documental 4

Procedencia y alcance de las evidencias . . . 4

Archivos técnicos adjuntos . . . 4

2. Arquitectura y modelo de datos 5

Modelo entidad–relación de AUTH . . . 6

Comprobación de independencia . . . 6

3. Seguridad, contratos y método de detección 7

Flujo de decisión . . . 7

Claves y vigencia de JWT . . . 7

Matriz de permisos . . . 8

Límites operativos . . . 8

Contratos OpenAPI . . . 9

Método de análisis esteganográfico . . . 10

4. Matriz de riesgos y controles 13

Dependencias y entorno de desarrollo . . . 16

Relación con referencias del enunciado . . . 16

5. Pruebas y evidencias reproducibles 17

Resultados verificados el 05/11/2026 . . . 17

Cómo reproducir cada grupo . . . 17

Trazabilidad de escenarios . . . 19

Resultados registrados de integración . . . 19

Indicadores reales de las cinco muestras . . . 20

Ejemplos de auditoría depurada . . . 22

6. Preparación, ejecución y operación 24

Preparación e inicio . . . 24

Uso . . . 25

Operación independiente . . . 25

Pruebas reproducibles . . . 25

Entregables . . . 26

7. Guion de sustentación con Postman **27**

¿Cumplimos la independencia? . . . 27
Preparación . . . 27
T01. Independencia de los servicios . . . 29
T02. Contraseña correcta e incorrecta . . . 29
T03. Sin JWT, con JWT alterado y con JWT vencido . . . 30
T04. Usuario autenticado sin permiso . . . 30
T05. Crear cuentas y mostrar la relación de muchos a muchos . . . 30
T06. Intentar duplicar una asignación . . . 31
T07. Retirar el rol con el mismo JWT . . . 31
T08. Dejar AUTH inaccesible . . . 32
T09. Analizar la imagen de referencia y la imagen con carga . . . 32
T10. Archivo inválido, corrupto y excesivo . . . 32
T11. Consultar un resultado propio y otro ajeno . . . 33
T12. Revisar la auditoría sin mostrar secretos . . . 33
Anexo B. La decisión interna . . . 34
Anexo C. El resultado del análisis . . . 34
Para qué sirve cada solicitud . . . 35

Anexo A. Mapa de cumplimiento **38**

2.1 Arquitectura e independencia . . . 38
2.2 AUTH . . . 38
2.3 STEGO . . . 38
2.4 Comunicación segura y RBAC . . . 39
2.5 Contrato mínimo . . . 39
3 Entregables y presentación . . . 40
Cobertura del documento completo . . . 41

Anexo B. Referencia de las APIs OpenAPI **42**

AUTH - AUTH Backend V1 . . . 42
Modelos de datos de AUTH . . . 49
STEGO - STEGO Backend V1 . . . 53
Modelos de datos de STEGO . . . 55

Anexo C. Inventario documental y archivos adjuntos **56**

Referencias técnicas **57**

1. Presentación y alcance documental

Este informe reúne los entregables documentales del Backend V1 de la aplicación de chat seguro: arquitectura, identidad N:M, JWT, RBAC vigente, contratos REST, método de detección esteganográfica, matriz de riesgos y evidencias reproducibles. Incluye además las instrucciones de ejecución y el guion de sustentación.

La primera versión comprende dos microservicios autónomos: AUTH y STEGO. La interfaz gráfica, las salas, la mensajería en tiempo real, el cifrado de extremo a extremo y la restricción de dispositivos corresponden a etapas posteriores.

Procedencia y alcance de las evidencias

El contenido se consolida desde los documentos vigentes 00 a 05 de App/docs, sus diagramas, contratos y archivos de pruebas, junto con el README de App. El directorio historico conserva etapas anteriores y no se usa como descripción principal de esta versión.

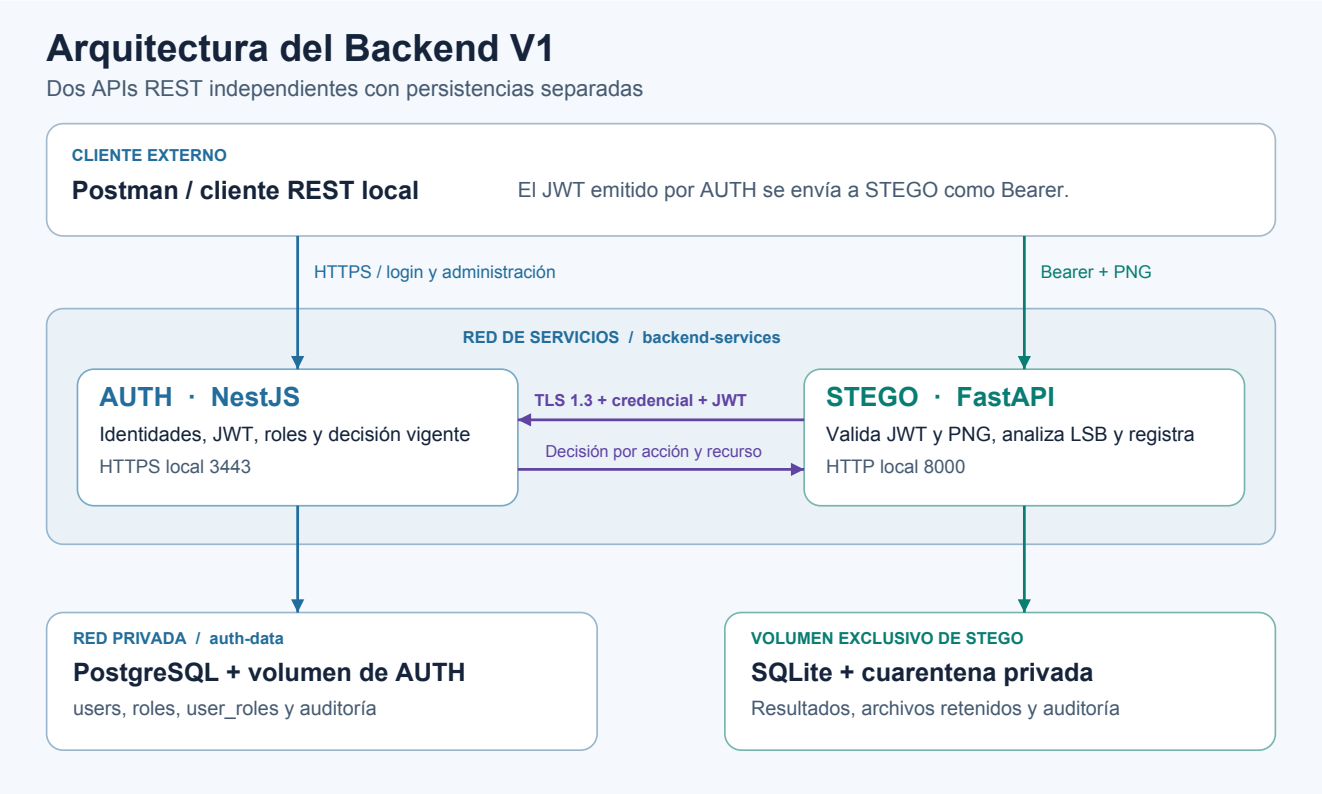
Los resultados de pruebas se transcriben de los registros existentes con revisión documentada al 05/11/2026. La elaboración de este informe no constituye una nueva ejecución de los servicios ni de sus pruebas. La sustentación presencial y la entrega del repositorio corresponden al equipo.

Archivos técnicos adjuntos

El PDF incorpora como archivos adjuntos los Markdown de base, los SVG, ambos contratos OpenAPI, la colección REST, las cinco imágenes de prueba, el manifiesto y los registros de resultados. Se conservan sus nombres y rutas de origen dentro del proyecto. Pueden extraerse con un lector PDF que admita archivos adjuntos. Los contratos JSON son las especificaciones completas importables; su consulta legible figura también en el anexo de APIs.

La carátula toma la identidad institucional, los autores y el docente de la referencia proporcionada. El título y la fecha de entrega corresponden a la consigna del proyecto; la revisión documental se identifica por separado.

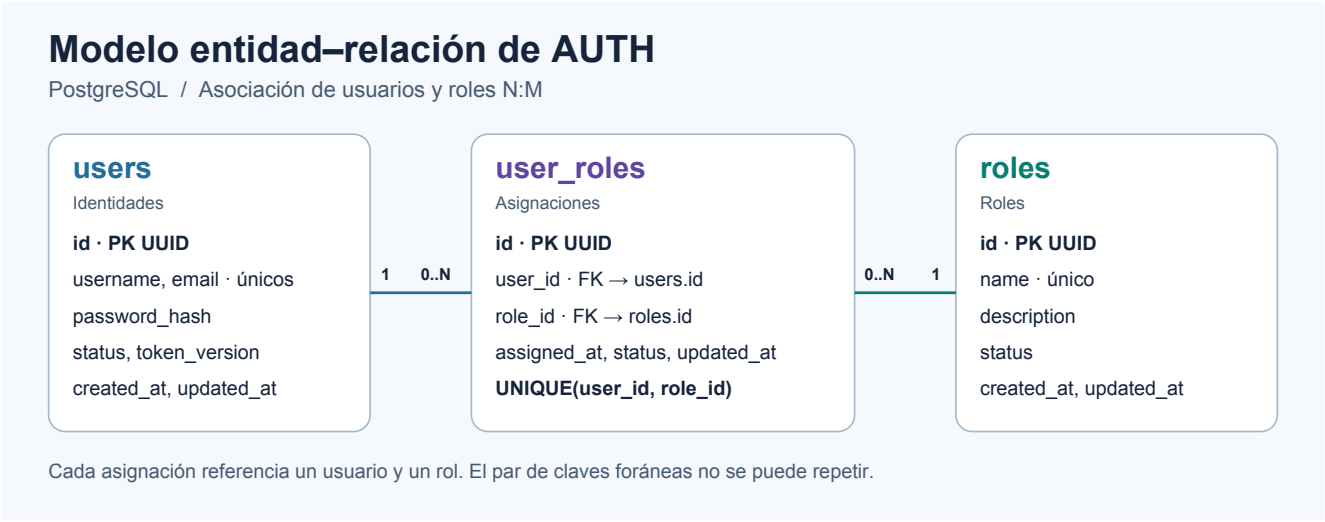
2. Arquitectura y modelo de datos



Arquitectura de AUTH y STEGO

AUTH y STEGO se ejecutan como procesos y contenedores separados, con sus propias rutas, configuración y guías. NestJS permite reutilizar la base del taller y organizar validación, lógica y persistencia. FastAPI permite manejar el contrato REST junto a Pillow, NumPy y SciPy. SQLite mantiene los resultados y la propiedad de los análisis en STEGO. PostgreSQL mantiene las identidades y la auditoría de AUTH en la red privada `auth-data`.

Modelo entidad–relación de AUTH



Relación N:M entre users, user_roles y roles

Un usuario puede recibir cero o más asignaciones y un rol puede figurar en cero o más asignaciones. Cada asignación apunta a un usuario y un rol mediante claves foráneas obligatorias. `user_roles.id` identifica la asignación y `UNIQUE(user_id, role_id)` impide duplicarla. El diagrama destaca los campos exigidos por la consigna y los campos que intervienen en el control de sesión. La API usa algunos nombres en español y la base conserva los nombres físicos que aparecen en la imagen.

- Entidades User, Role y UserRole.
- Migración del modelo AUTH V2.
- Docker Compose del conjunto, guía AUTH y guía STEGO.

Comprobación de independencia

Desde la carpeta `App`, `node scripts/compose.cjs stop stego` permite seguir consultando la salud de AUTH. `node scripts/compose.cjs start stego` vuelve a iniciar su contenedor. Si se detiene AUTH, la salud de STEGO sigue respondiendo y su ruta de análisis devuelve 503. El guion de sustentación indica el orden de estas operaciones.

3. Seguridad, contratos y método de detección

Flujo de decisión

1. AUTH verifica credenciales con Argon2id y emite JWT RS256 de vigencia acotada. La privada RSA permanece en AUTH.

2. STEGO verifica localmente firma RS256, emisor, audiencia, expiración, tipo de acceso, identificador UUID, versión y duración del token. No usa roles enviados por el cliente.

3. Antes de leer el cuerpo multipart, STEGO consulta `POST https://auth:3443/internal/authorization/check`, enviando JWT y acción, con `X-Service-Token`. El certificado se verifica con la CA y TLS mínimo 1.3.

4. AUTH vuelve a comprobar sesión, usuario, roles y asignaciones activos desde PostgreSQL. Para lectura, STEGO aporta el propietario obtenido de su SQLite, nunca del cliente.

5. STEGO exige que la respuesta coincida en sujeto, acción y recurso y que `allowed` sea un booleano real. No almacena decisiones como permisos permanentes. Denegación → 403; JWT/sesión inválidos → 401; indisponibilidad de AUTH → 503; decisión mal formada → 502. No procesa el archivo sin permiso comprobado.

6. Una vez autorizado, valida el archivo, ejecuta el trabajador aislado y conserva métricas reales, SHA-256, propietario, fecha y política. La consulta posterior repite la autorización por recurso.

El endpoint interno requiere TLS real y la identidad de servicio; un JWT del usuario por sí solo no lo habilita. La credencial compartida identifica al servicio; no se presenta como mTLS. El cliente STEGO valida el certificado servidor y no sigue redirecciones. La ventana entre la decisión y la operación no desaparece: la revocación se hace efectiva en la siguiente decisión, sin prometer cancelar trabajos ya autorizados.

Claves y vigencia de JWT

AUTH firma con RS256 y un par RSA de al menos 3072 bits. El plazo inicial es 900 segundos y se configura con `JWT_EXPIRES_IN_SECONDS`. La firma vincula `sub` con el UUID del usuario e incluye `iss`, `aud`, `iat`, `exp`, `typ` y `tokenVersion`. Los roles que devuelve el login se consultan para informar al cliente. El permiso se decide de nuevo con los datos persistidos.

El inicializador mantiene el material local fuera de Git y distribuye a STEGO la clave pública, la CA y su credencial de servicio. La privada de firma y la clave privada del certificado TLS permanecen en AUTH. Al rotar el par de firma, actualizar la pública que usa STEGO, reiniciar ambos servicios y emitir JWT nuevos. Configuración de AUTH, módulo JWT y verificación en STEGO.

Matriz de permisos

Operación	ADMIN	ANALYST	USER
Login con credenciales válidas	Sí	Sí	Sí
Perfil y logout propios	Sí	Sí	Sí
CRUD de usuarios y roles; asignar/retirar roles	Sí	No	No
Analizar un PNG	Sí	Sí	No
Consultar resultados propios	Sí	Sí	No
Consultar resultados ajenos	Sí	No	No
Endpoint interno de decisiones	Solo cliente de servicio autenticado; vuelve a evaluar al usuario	Igual	Igual

La acumulación de roles aplica los permisos efectivos actuales, sin elevar a USER por enviar cabeceras falsas. Un ANALYST que pierde su asignación deja de poder analizar con el mismo JWT. El logout incrementa la versión de tokens del usuario; este mecanismo invalida sus tokens previos, no solamente un token individual.

Límites operativos

Los puertos de desarrollo se restringen a loopback. El canal interno usa HTTPS; STEGO expone HTTP local, por lo que el despliegue remoto requiere TLS de entrada. La red de servicios permite salida para operar; el trabajador de imágenes tiene las llamadas de red denegadas con seccomp. El contenedor no es privilegiado, no tiene capacidades y usa sistema raíz de solo lectura. El trabajador comparte el UID y el volumen de STEGO: este aislamiento limita recursos y red, pero no constituye una frontera completa frente a una vulnerabilidad que permita leer archivos del propio servicio. Mantener dependencias actualizadas y evaluar un aislamiento de filesystem más estricto para producción.

Cada instancia STEGO está limitada a dos trabajos; SQLite y el volumen local corresponden a esta entrega de una sola instancia. Las credenciales y certificados necesitan gestión y rotación; no se incluye una plataforma de secretos ni alta disponibilidad.

Contratos OpenAPI

Servicio	Contrato entregable	Swagger local	Rutas
AUTH	OpenAPI AUTH	`https://localhost:3443/docs`	REST bajo `/api/v1`; la decisión interna queda en `/internal/authorization/check`
STEGO	OpenAPI STEGO	`http://localhost:8000/docs`	`/health` y operaciones bajo `/api/v1/steganography`

Los JSON son los contratos completos, importables por separado. En AUTH, `page` y `limit` son opcionales, con valores iniciales 1 y 20. Los esquemas documentan resultados, páginas, asignaciones y el formato del filtro global de errores. STEGO documenta el cuerpo multipart y los errores, incluida la respuesta con `analysis_id` cuando se registra un análisis fallido. En una operación protegida, 401 indica que no se autenticó la solicitud, 403 que la sesión carece de permiso, 502 que la decisión de AUTH no se pudo verificar y 503 que el servicio no pudo completar la autorización o el procesamiento.

Respuestas para la demostración

FastAPI genera el contrato de STEGO en `/openapi.json` y lo muestra en Swagger UI en `/docs`. AUTH usa NestJS para servir su contrato y su Swagger. Postman usa una colección de solicitudes guardadas para encadenar login, análisis y consulta durante la sustentación. Swagger y los JSON exportados comparten las definiciones del código. Los ejemplos de Postman provienen del contrato. El contrato de STEGO agrupa sus dos operaciones de negocio en **Esteganografía** y `GET /health` en **Operación**. La decisión interna se documenta en AUTH y se ejecuta desde STEGO.

Operación	Éxito y tipo de cuerpo	Rechazos que muestra la colección
Login y consultas de AUTH	`200` con JSON	`400` si corresponde a validación, `401` por credenciales o sesión, `403` por permisos y `404` si aplica al recurso.
Creación de usuario, rol o asignación	`201` con JSON del recurso creado	`400` entrada inválida, `401` sesión, `403` permiso y `409` conflicto. Al asignar también aplica `404` por usuario o rol inexistente.
Actualización de usuario o rol	`200` con JSON actualizado	`400`, `401`, `403`, `404` y `409`.
Retiro de asignación, eliminación o logout	`204` sin cuerpo	Los errores mantienen el JSON de AUTH con `statusCode`, `message`, `path` y `requestId` cuando está disponible.
Analizar imagen con ANALYST o ADMIN	`200` con el análisis	`400` petición o PNG corrupto, `401` autenticación, `403` permiso, `413` tamaño, `415` formato, `502` decisión no verificable y `503` indisponibilidad.

Operación	Éxito y tipo de cuerpo	Rechazos que muestra la colección
Consultar resultado	`200` con el análisis	`400` ID inválido, `401` autenticación, `403` permiso o propiedad, `404` resultado inexistente, `502` y `503` para fallas de autorización.

AUTH responde `429` cuando supera el límite y las APIs documentan `500` en sus operaciones protegidas para un fallo interno. STEGO devuelve `{"detail": "..."}` para los rechazos previos al análisis. Si el procesamiento alcanzó a registrar un error, la respuesta incluye también `analysis_id` y los datos del registro con `status: error`. La consulta interna de AUTH responde `200` con `allowed: true` o `false` cuando la decisión fue válida. STEGO traduce la denegación en `403` al cliente.

Regeneración desde `App/ms-users` con `npm run docs:openapi`. Desde `App/ms-stego`, con el entorno Python de pruebas activo, ejecutar `python -m scripts.export_openapi`. Ambas herramientas escriben en `App/docs/contratos`.

Método de análisis esteganográfico

Alcance y fundamento

Se selecciona PNG de 8 bits en escala de grises, RGB o RGBA (alfa excluido). No se admiten PNG indexados, de 16 bits, animados ni JPEG/BMP. Cumple la alternativa PNG del enunciado. La técnica busca equiprobabilidad entre valores adyacentes `2j` y `2j+1`, un efecto compatible con reemplazar sus bits menos significativos con una carga de bits aproximadamente balanceados.

El análisis de pares de valores se inspira en Westfeld y Pfitzmann, Attacks on Steganographic Systems. Esta implementación usa un **contraste Pearson condicional con ambas categorías por pareja**; no reproduce exactamente la fórmula y los grados de libertad de aquel detector. La identificación de técnica es `pov_pearson_lsb_png8_v1`; versión de cálculo `pov-pearson-v1`.

Cálculo implementado

1. Validar extensión `.png`, MIME `image/png`, firma PNG e IHDR. Comprobar tamaño, profundidad, canales y dimensiones antes de decodificar. Pillow verifica estructura e integridad y vuelve a abrir para cargar píxeles.

2. Extraer muestras de intensidad de los canales admitidos. Construir el histograma `h[0..255]`.

3. Para cada pareja `j`, definir `e_j = h[2j]`, `o_j = h[2j+1]`, `n_j = e_j + o_j`. Incluir solo parejas con `n_j ≥ 10`: bajo la hipótesis de equiprobabilidad, cada frecuencia esperada es `n_j/2 ≥ 5`.

4. Exigir al menos 8192 muestras de canal y 64 parejas elegibles. Si falta evidencia, devolver `no_concluyente`, sin inventar un p-valor.

5. Bajo `H0: P(par | pareja) = P(impar | pareja) = 1/2`, calcular:

```
X² = Σ [ (e_j - n_j/2)²/(n_j/2) + (o_j - n_j/2)²/(n_j/2) ]
   = Σ (e_j - o_j)²/n_j
k  = número de parejas elegibles
gl = k
p  = P(Chi-cuadrado(gl) ≥ X²) = scipy.stats.chi2.sf(X², k)
```

Cada pareja tiene un total condicionado y dos categorías: aporta un grado de libertad. No se ajusta una probabilidad libre desde los datos; la proporción 1/2 se fija en H0. Se usa la función de supervivencia chi2 de SciPy. La aproximación supone muestras suficientemente independientes; la correlación espacial real limita su interpretación.

6. Clasificar mediante parámetros propios, explícitos y no impuestos por el PDF:

Condición	Estado	Justificación
Muestra/parejas insuficientes	`no_concluyente`	La aproximación no tiene el soporte mínimo adoptado
`p ≥ 0,05`	`sospechoso`	Los datos son compatibles con equiprobabilidad; indicio heurístico
`p < 0,01`	`sin_indicios`	Se rechaza fuertemente esa hipótesis concreta; no demuestra ausencia de mensajes
`0,01 ≤ p < 0,05`	`no_concluyente`	Zona intermedia conservadora
Decodificación, aislamiento o procesamiento fallidos	`error`	No se obtuvo un análisis válido

Los valores 0,05/0,01 son umbrales iniciales de decisión, no umbrales calibrados sobre un corpus representativo. Los mínimos 8192/64/10 buscan evitar conclusiones con pocos datos, sin garantizar independencia ni precisión. **Un p-valor alto no es la probabilidad de que exista esteganografía** y no aceptar/rechazar H0 equivale a demostrar un mensaje.

Además se informa `lsb_one_fraction`, la fracción observada de bits bajos en uno; no determina por sí sola el estado. Todos los indicadores salen de los píxeles, no del nombre de archivo, hash, encabezado del mensaje o una etiqueta prefijada. El detector no conoce el formato de carga usado por el generador de pruebas.

Construcción y resultados de muestras

ZZTOKEN0ZZ crea un control sintético de 256 × 256 RGB con intensidades pares. Para `oculta_lsb.png` inserta realmente un encabezado `STEGOv1!`, longitud y mensaje UTF-8, completando capacidad con bits pseudoaleatorios de semilla fija 4201. Recupera los bits y comprueba el mensaje por aserción. Otra muestra inserta solo el encabezado y mensaje, sin llenar la capacidad; una tercera usa ruido benigno independiente con semilla 4202 y sin mensaje. La pequeña es 8 × 8.

Archivo	Verdad de construcción	Resultado calculado	Interpretación
`original_control.png`	Sin carga	`sin_indicios`	Control negativo sintético
`oculta_lsb.png`	Carga LSB de alta ocupación	`sospechoso`	Verdadero positivo del caso compatible
`oculta_baja_carga.png`	Mensaje LSB de baja ocupación	`sin_indicios`	Falso negativo: limitación real
`ruido_sin_mensaje.png`	Ruido sin mensaje	`sospechoso`	Falso positivo: equiprobabilidad benigna
`pequena.png`	Sin carga	`no_concluyente`	Evidencia insuficiente

El manifiesto contiene hashes e indicadores reproducibles. La ejecución por API con contenedores reales comprueba las cinco muestras, incluyendo aislamiento Linux. Entre las cuatro muestras concluyentes hay TP=1, FP=1, TN=1 y FN=1; una adicional es no concluyente. Son ejemplos construidos para mostrar comportamiento, no un corpus independiente ni una estimación de precisión general. No se afirma TPR ≥ 90 %, FPR ≤ 5 % ni las latencias de documentos de actividades anteriores.

Política, límites y controles

`sin_indicios` → advertencia y eliminación del archivo; demás estados → cuarentena privada durante siete días. Los errores previos a admitir el PNG se rechazan sin análisis. La cuarentena es una medida preventiva, no un dictamen de contenido ilícito. Se conserva SHA-256 y metadatos del análisis.

La baja carga puede evadir el contraste. Ruido benigno, cifrado previo de píxeles u otras distribuciones balanceadas pueden producir falsos positivos. No identifica mensajes específicos, LSB matching, esteganografía adaptativa, contenido oculto solo en metadatos, JPEG ni todas las variantes de PNG. Las relaciones espaciales y el tratamiento conjunto de canales también limitan el modelo. El detector es una técnica verificable de alcance acotado, no una herramienta forense concluyente.

El PNG se trata como entrada hostil: validación, límites de píxeles, proceso separado, CPU/memoria/tiempo, entorno sin credenciales y seccomp sin red. Se consideran las recomendaciones de seguridad y descompresión de Pillow. Esto reduce riesgos, sin prometer protección completa frente a fallos de las bibliotecas de decodificación.

4. Matriz de riesgos y controles

Alcance: dos APIs AUTH/STEGO, identidad PostgreSQL, resultados SQLite, imágenes y comunicación interna. Esta matriz vincula riesgos con controles implementados y evidencia; no atribuye probabilidades, puntuaciones residuales ni certificación que no se hayan medido. Los riesgos restantes requieren seguimiento.

M01 - Credenciales e identidades

Campo	Detalle
Amenaza	Suplantación, fuerza bruta, divulgación
Vulnerabilidad o condición	Contraseñas débiles, entradas inválidas o respuestas que divulguen datos
Riesgo y efecto	Acceso indebido y exposición de usuarios
Controles implementados	Argon2id; DTO validados; límites de login; hashes excluidos de respuestas; secretos fuera de Git
Evidencia	AUTH: pruebas de credenciales/validación; T02; auditoría depurada
Principios aplicados	Zero Trust: autenticar explícitamente. Secure by Design: hash y validación incorporados. Shift Left: pruebas de login/DTO
Riesgo restante	Robo de contraseña en el cliente; rate limiting en memoria no coordina réplicas

M02 - Asignaciones N:M y permisos

Campo	Detalle
Amenaza	Elevación de privilegios
Vulnerabilidad o condición	Confiar en roles obsoletos o admitir pares duplicados
Riesgo y efecto	Analizar o administrar después de perder el permiso
Controles implementados	FK, UUID propio y UNIQUE del par; usuarios/roles/asignaciones activos consultados en AUTH; revocación con el mismo JWT; ADMIN para administración
Evidencia	T04-T07; pruebas AUTH N:M y acceso administrativo
Principios aplicados	Zero Trust: permiso vigente por operación. Secure by Design: invariantes en base. Shift Left: duplicación y revocación automatizadas
Riesgo restante	Una cuenta ADMIN comprometida conserva amplias facultades; cambios no cancelan trabajos ya autorizados

M03 - JWT y claves

Campo	Detalle
Amenaza	Falsificación, alteración o fuga
Vulnerabilidad o condición	Algoritmos ambiguos, claims no verificados o clave privada compartida
Riesgo y efecto	Suplantar identidades entre servicios
Controles implementados	RS256 explícito; RSA 3072; verificación firma/emisor/audiencia/expiración/tipo/versión; privada solo AUTH; logout invalida versión
Evidencia	T03; prueba de logout; prueba de ausencia de privada en STEGO
Principios aplicados	Zero Trust: no confiar en el token sin verificar. Secure by Design: separación firmante/verificador. Shift Left: tokens alterados/vencidos
Riesgo restante	Robo de un token vigente; rotación manual de claves y TLS local de corta duración

M04 - Comunicación AUTH-STEGO

Campo	Detalle
Amenaza	Suplantación de servicio, interceptación o decisión falsa
Vulnerabilidad o condición	Canal sin verificar o decisión reutilizada/deformada
Riesgo y efecto	Autorizar análisis sin permiso real
Controles implementados	HTTPS TLS ≥1.3 y CA verificada; credencial de servicio; sin redirecciones; respuesta estricta ligada a sujeto/acción/recurso; consulta por petición; denegación ante error
Evidencia	T08; pruebas unitarias de timeout, respuesta inválida y denegada; pruebas AUTH del guard de servicio
Principios aplicados	Zero Trust: dos identidades y verificación continua. Secure by Design: fallo seguro. Shift Left: simulación de fallos
Riesgo restante	Una fuga de credencial exige rotación; caída de AUTH impide operaciones protegidas; no hay alta disponibilidad

M05 - Archivos y proceso de imagen

Campo	Detalle
Amenaza	Carga maliciosa, descompresión excesiva, explotación del decodificador
Vulnerabilidad o condición	PNG corrupto, dimensiones excesivas, nombres manipulables o recursos sin límite
Riesgo y efecto	Denegación de servicio o compromiso del análisis
Controles implementados	Extensión/MIME/firma/IHDR/Pillow; archivo 5 MiB y píxeles limitados; nombres UUID; no ejecutar archivos; trabajador separado con entorno sin credenciales, CPU/memoria/tiempo, seccomp sin red; contenedor sin root/capacidades y raíz de solo lectura
Evidencia	T09-T10; prueba real de socket EPERM y UID no root; límites configurados en código/Compose
Principios aplicados	Zero Trust: archivo hostil aun autenticado. Secure by Design: límites y aislamiento. Shift Left: archivos corruptos y excesivos
Riesgo restante	Vulnerabilidades desconocidas de Pillow/NumPy/SciPy; trabajador comparte UID y volumen STEGO, sin frontera completa de filesystem

M06 - Resultados, cuarentena y registros

Campo	Detalle
Amenaza	Lectura ajena, manipulación de rutas o fuga por logs
Vulnerabilidad o condición	Confiar en propietario enviado por cliente, servir imágenes o registrar tokens
Riesgo y efecto	Divulgación de resultados, secretos o imágenes
Controles implementados	Propietario desde SQLite; autorización por recurso en AUTH; resultados propios ANALYST/todos ADMIN; no descarga pública; archivos privados; cuarentena siete días y purga; auditoría con request_id sin tokens/contraseñas
Evidencia	T11-T12; prueba de retención; persistencia tras reinicio
Principios aplicados	Zero Trust: verificar acceso al recurso. Secure by Design: datos privados por defecto. Shift Left: prueba de lectura ajena y secretos en auditoría
Riesgo restante	Administradores del host acceden a volúmenes; no hay cifrado de volumen implementado; retención de metadatos por acordar para producción

M07 - Decisiones del detector

Campo	Detalle
Amenaza	Evasión o interpretación exagerada
Vulnerabilidad o condición	Técnica de alcance limitado, umbrales sin corpus representativo, ruido benigno
Riesgo y efecto	Falsos negativos/positivos y decisiones incorrectas
Controles implementados	Métricas reales, no_concluyente, advertencia/cuarentena preventiva, muestras de baja carga y ruido, limitaciones explícitas; no afirmar prueba concluyente
Evidencia	Manifiesto: TP=1 FP=1 TN=1 FN=1 y una no concluyente; T09 real; tests del cálculo
Principios aplicados	Secure by Design: estados y límites explícitos. Shift Left: controles positivos/negativos y pruebas de evasión. Zero Trust: el resultado no concede permisos ni confianza permanente
Riesgo restante	No detecta todos los mensajes/técnicas; falta calibración independiente antes de uso operativo

Dependencias y entorno de desarrollo

La revisión del 05/11/2026 actualizó las dependencias de ejecución y verificó `npm audit --omit=dev`. Las herramientas de desarrollo usan versiones fijadas en el lockfile, con overrides de `glob` y `minimatch` verificados mediante build, lint, Jest y las pruebas de integración. Se retiró el auxiliar cloud de despliegue que no usa esta entrega.

La herramienta de pruebas Jest mantiene una dependencia transitiva de `deepmerge` con un aviso de severidad baja sin versión corregida publicada. Se conserva su configuración dentro del código del repositorio y no se aceptan configuraciones de Jest de terceros. Este aviso del entorno de desarrollo se distingue de los paquetes instalados en el contenedor de ejecución. Aviso del mantenedor. Los avisos de `body-parser` y `multer` se atendieron actualizando las versiones de ejecución. body-parser · multer.

Relación con referencias del enunciado

• **Zero Trust:** verificación explícita de usuario y servicio, decisión vigente por petición, mínimo privilegio y denegación por defecto; se aplica como principio de NIST SP 800-207. La red interna no reemplaza la autenticación.

• **Secure by Design:** separación de identidades/datos/claves; contratos estrictos; integridad N:M; controles de entrada y límites de proceso forman parte de la arquitectura.

• **Shift Left Security:** pruebas de credenciales, claims, permisos, fallos de autorización, archivos y cálculo se incluyen junto al código y se ejecutan antes de entregar. No se declara una canalización CI ni SAST que no estén implementados.

• **ISO/IEC 27001:** la identificación de activos/riesgos, selección de controles, gestión de accesos, protección de secretos y auditoría se relacionan conceptualmente con la gestión de seguridad del estándar citado por el PDF. Esta matriz es evidencia técnica del proyecto; no constituye un SGSI completo, auditoría normativa ni certificación. Referencia oficial.

• **ISO/IEC 27034-1:** requisitos de seguridad ligados al diseño, implementación y validación de la aplicación; M01-M07 enlazan controles y pruebas a riesgos. No se afirma cumplimiento integral ni certificación. Referencia oficial.

La política de carga también se fundamenta en la guía OWASP File Upload: controles complementarios de formato, tamaño, nombres y almacenamiento. La verificación de JWT sigue el propósito de RFC 7519 y RFC 8725. No se confunden las referencias con resultados medidos.

5. Pruebas y evidencias reproducibles

Ejecutar los comandos de App/README. La suite de integración construye dos imágenes independientes y utiliza PostgreSQL y SQLite reales, volúmenes nuevos y puertos temporales; se limpian solo esos recursos al finalizar. Las llamadas asíncronas de las pruebas unitarias de fallos utilizan respuestas simuladas para explorar decisiones inválidas sin modificar AUTH.

Resultados verificados el 05/11/2026

• STEGO: **16 casos unitarios aprobados**, incluidos cálculo, carga LSB recuperable, retención, OpenAPI, siete formas de fallo/denegación de AUTH y rechazo antes de leer el cuerpo. Salida.

• Conjunto AUTH+STEGO: **15 comprobaciones integradas aprobadas**, que cubren los doce escenarios T01-T12 agrupando T05/T06 y añadiendo aislamiento, logout y persistencia. Resultado con fecha de ejecución UTC.

• AUTH: 7 pruebas unitarias con Jest y 34 pruebas funcionales/seguridad, reejecutadas con el ejecutor de Node.js y PostgreSQL temporal para los casos funcionales y de seguridad.

• Resultados completos por API de las cinco muestras, verdad de construcción y métricas y auditoría sin credenciales.

La colección simplificada se verifica con Postman CLI y bases temporales mediante el ejecutor versionado. El registro del último ensayo conserva su salida. La versión JSON ejecutó 22 solicitudes y 34 aserciones sin fallos. La versión YAML ejecutó el flujo de login, análisis y consulta. Las comprobaciones incluyeron análisis con ADMIN y ANALYST, rechazo con USER o JWT ausente, lectura ajena, entrada inválida, creación y eliminación con cuerpo vacío. También se compararon los contratos servidos por Swagger con ambos archivos OpenAPI exportados.

La fecha UTC puede pertenecer al día siguiente de la fecha local Ecuador (UTC−5). Los UUID/horas cambian al repetir la suite; los hashes y cálculos de las imágenes generadas son reproducibles con las dependencias fijadas. Los usuarios de prueba se crean en una base temporal; los UUID publicados en evidencia no identifican cuentas del despliegue normal.

Cómo reproducir cada grupo

La consigna especifica escenarios y tipos de prueba. El equipo usa las herramientas que corresponden a cada aplicación.

Grupo	Código y herramientas	Comando
Unitarias AUTH	Jest en validación y controlador	Carpeta `App/ms-users`, comando `npm test -- --runInBand`
Funcionales y seguridad AUTH	Suite Node.js y ejecutor con PostgreSQL temporal	Carpeta `App/ms-users`, comando `npm run test:docker`
Unitarias y seguridad STEGO	pytest en núcleo y decisiones de autorización	Carpeta `App/ms-stego`, comando `python -m pytest -q` con el entorno de prueba activo

Grupo	Código y herramientas	Comando
Integración del conjunto	Runner Docker y solicitudes HTTPX	Carpeta `App`, comando `ms-stego/.venv/Scripts/python.exe tests/run.py` en Windows
Ensayo manual	Colección REST y guion Postman	Reutilizar **Analizar imagen** y **Consultar resultado** cambiando el JWT o el archivo según el guion

Los archivos de auditoría de ejemplo usan identidades sintéticas. La muestra de AUTH conserva un registro de la fase previa; la de STEGO corresponde a la última ejecución del conjunto. Los comandos reproducen los casos con bases temporales. Las instrucciones de instalación del entorno están en la guía de ejecución.

Para repetir también las 22 solicitudes de prueba y 34 aserciones de Postman, instalar Postman CLI y ejecutar desde `App`:

```
ms-stego/.venv/Scripts/python.exe tests/run.py --postman
```

El ejecutor prepara un proyecto temporal, guarda `postman-coleccion.txt` y luego ejecuta T01–T12. Si existe la copia YAML de `postman/collections`, también prueba su flujo de login, análisis y consulta. Usa la CA local y mantiene la verificación TLS. Los resultados del caso de prueba pueden cambiar de UUID y hora al repetirlo.

Trazabilidad de escenarios

Escenario PDF	Comprobación ejecutada	Riesgos relacionados
T01	Detener/iniciar STEGO sin interrumpir AUTH; detener/iniciar AUTH manteniendo salud STEGO durante T08	M04
T02	Login válido devuelve JWT; contraseña incorrecta → 401	M01
T03	JWT ausente, inválido, alterado y vencido → 401 sin fila nueva de análisis	M03
T04	USER → 403 incluso con cabeceras falsas ADMIN	M02
T05	Usuario USER+ANALYST y ANALYST compartido por usuarios; persistencia N:M preparada por seed y contrastada con API	M02
T06	Repetición del par usuario/rol → 409; la prueba AUTH previa también comprueba la asignación	M02
T07	Retirar ANALYST, reutilizar el mismo JWT y comprobar 403 sin análisis; restaurar el rol	M02
T08	Detener AUTH; STEGO /health responde y carga devuelve 503 sin fila nueva; reiniciar AUTH	M04
T09	Control y carga LSB real → indicadores y estados distintos; medir también ruido, baja carga y muestra insuficiente	M05, M07
T10	Firma falsa, extensión/MIME incompatibles, PNG truncado, archivo >5 MiB y dimensiones excesivas → rechazo apropiado	M05
T11	ANALYST propietario → 200; otro ANALYST → 403; ADMIN → 200	M06
T12	Auditoría incluye peticiones denegadas, indisponibilidad y analysis_id; no contiene tokens completos, contraseña ni credencial de servicio	M01, M03, M06

Adicionalmente: prueba dentro del contenedor de UID no root, ausencia de variables DB_ y clave privada JWT, y creación de socket denegada tras activar seccomp; logout invalida el mismo JWT ante STEGO; reinicio STEGO conserva resultados. Las pruebas unitarias de autorización rechazan timeout, JSON inválido, booleano textual, sujeto/recurso incorrectos, 401 y denegación explícita. La prueba del middleware confirma que no se lee el archivo ni se invoca la aplicación cuando falta permiso.

No se ha ejecutado una evaluación estadística independiente, una prueba de carga masiva o una certificación de seguridad. La sustentación en clase y la publicación/entrega del repositorio corresponden al equipo; las evidencias locales no las sustituyen.

Resultados registrados de integración

Ejecución registrada: 2026-11-05T05:22:03.618853+00:00 (UTC). Dos imágenes y procesos reales, PostgreSQL 18 y SQLite separados; puertos y volúmenes temporales.

Comprobación	Resultado
T01 APIs y procesos autónomos	Aprobada
Aislamiento: no root, sin BD AUTH ni clave privada y socket denegado	Aprobada
T02 contraseña inválida denegada	Aprobada

Comprobación	Resultado
T03 JWT ausente manipulado y vencido sin análisis	Aprobada
T04 USER denegado aun con cabecera de rol falsa	Aprobada
T05 T06 N:M persistida y duplicado rechazado	Aprobada
T09 análisis real de control y carga LSB con sandbox	Aprobada
T10 firma tipo extensión corrupción tamaño y dimensiones	Aprobada
T11 propiedad de resultados aplicada	Aprobada
UUID de otra versión rechazado sin reescribir el recurso	Aprobada
T07 revocación inmediata con mismo JWT	Aprobada
T08 AUTH caído no ejecuta análisis	Aprobada
Logout entre microservicios invalida mismo JWT	Aprobada
T12 auditoría depurada de accesos y análisis	Aprobada
Persistencia tras reiniciar STEGO	Aprobada

Indicadores reales de las cinco muestras

original_control.png

Campo	Valor registrado
Estado / política	sin_indicios / advertencia
SHA-256	b7894c8b99be99ea0b3d88dc5deaa89ff1ed627cbf7db74148ab2a24f967afda
Muestras / parejas elegibles	196608 / 128
Chi-cuadrado / grados de libertad	196608.0 / 128
p de equiprobabilidad	0.0
Fracción LSB en uno	0.0
Registro UTC	2026-11-05T05:21:34.083678+00:00

oculta_lsb.png

Campo	Valor registrado
Estado / política	sospechoso / cuarentena
SHA-256	212985d37f143c8bcdd2a4cb276062f53687de556ac5ad39fca0f0f2ffa4551d
Muestras / parejas elegibles	196608 / 128
Chi-cuadrado / grados de libertad	125.296875 / 128
p de equiprobabilidad	0.5510812305839959
Fracción LSB en uno	0.5005086263020834
Registro UTC	2026-11-05T05:21:34.956821+00:00

oculta_baja_carga.png

Campo	Valor registrado
Estado / política	sin_indicios / advertencia
SHA-256	10f50eaed2346139855eda2391ab67d6703fb9f6748b5007acde3599e3e411e2
Muestras / parejas elegibles	196608 / 128
Chi-cuadrado / grados de libertad	195825.01562500003 / 128
p de equiprobabilidad	0.0
Fracción LSB en uno	0.0009969075520833333
Registro UTC	2026-11-05T05:21:35.836220+00:00

ruido_sin_mensaje.png

Campo	Valor registrado
Estado / política	sospechoso / cuarentena
SHA-256	9b0d38afacdaad5dcbc4e4f3ba47ae3778d9fd79a4b2a4196a778ee54e70604f
Muestras / parejas elegibles	196608 / 128
Chi-cuadrado / grados de libertad	110.73900491426063 / 128
p de equiprobabilidad	0.8619998264508512
Fracción LSB en uno	0.4999491373697917
Registro UTC	2026-11-05T05:21:36.735595+00:00

pequena.png

Campo	Valor registrado
Estado / política	no_concluyente / cuarentena
SHA-256	eceb6a804feb26f68fe45ec6b9dd9ea012e189c9e1f3ed41f40f19c3280ed5c5
Muestras / parejas elegibles	192 / 0
Chi-cuadrado / grados de libertad	No calculado / No aplica
p de equiprobabilidad	No calculado
Fracción LSB en uno	0.0
Registro UTC	2026-11-05T05:21:37.608425+00:00

Ejemplos de auditoría depurada

AUTH: 12 registros en el archivo adjunto.


```
{
  "request_id": "auth-v2-compose-check",
  "actor_id": null,
  "service_id": null,
  "method": "GET",
  "route": "/api/v1/auth/profile",
  "status_code": 401,
  "action": null,
  "allowed": null
}

{
  "request_id": "auth-v2-compose-check",
  "actor_id": "24025d08-3e89-4266-94fe-76b9b9e4231c",
  "service_id": null,
  "method": "POST",
  "route": "/api/v1/auth/logout",
  "status_code": 204,
  "action": null,
  "allowed": null
}

{
  "request_id": "auth-v2-compose-check",
  "actor_id": "7b1cd245-2095-4cdb-8f15-70290c97e007",
  "service_id": "STEGO",
  "method": "POST",
  "route": "/internal/authorization/check",
  "status_code": 200,
  "action": "steganography.analyze",
  "allowed": false
}
```


STEGO: 41 registros en el archivo adjunto.


```
{
  "id": 7,
  "occurred_at": "2026-11-05T05:21:01.597309+00:00",
  "request_id": "b4acec03-ecfe-47f8-833e-7352bb1be971",
  "user_id": "56838e3b-ac52-488c-8c76-5c9b84fe2879",
  "method": "GET",
  "route": "/api/v1/steganography/results/:id",
  "status_code": 403,
  "analysis_id": null,
  "event": "request"
}
```

```
{
  "id": 11,
  "occurred_at": "2026-11-05T05:21:02.113109+00:00",
  "request_id": "926a449d-bfa5-497c-9097-baec692b7abf",
  "user_id": "8d5540b5-e994-439b-9a46-e92948374421",
  "method": "POST",
  "route": "/api/v1/steganography/analyze",
  "status_code": 403,
  "analysis_id": null,
  "event": "request"
}

{
  "id": 12,
  "occurred_at": "2026-11-05T05:21:02.199764+00:00",
  "request_id": "7928ea0c-2e2a-43b9-b520-fc92f0f63be3",
  "user_id": null,
  "method": "POST",
  "route": "/api/v1/steganography/analyze",
  "status_code": 401,
  "analysis_id": null,
  "event": "request"
}
```

6. Preparación, ejecución y operación

Dos microservicios del primer parcial, dentro del único repositorio del equipo:

```
App/
├── ms-users/     AUTH: NestJS, TypeScript y PostgreSQL
├── ms-stego/     STEGO: FastAPI, Python y almacenamiento propio SQLite
├── docker-compose.yml
├── scripts/
├── tests/
└── docs/
```

AUTH administra identidades N:M, credenciales y permisos. STEGO recibe PNG, verifica el JWT, consulta la autorización vigente en AUTH y calcula indicadores estadísticos sobre los bits menos significativos. PostgreSQL y los volúmenes son infraestructura de apoyo.

Preparación e inicio

Requisitos: Git, Node.js 24 y npm, Docker Desktop con contenedores Linux y Docker Compose que admita `!override` para la suite de pruebas (2.24.4 o posterior). OpenSSL debe estar disponible; en Windows el inicializador busca también el incluido con Git. Python 3.12 solo se necesita en el host para ejecutar las pruebas Python, no para iniciar los contenedores.

Desde una copia nueva:

```
git clone https://github.com/arieleMM-dev/SecureShop_Lugma-a_Morillo_Villacres.git
cd SecureShop_Lugma-a_Morillo_Villacres/App/ms-users
npm ci
cd ..
node scripts/init-env.cjs
node scripts/compose.cjs up -d --build --wait
node scripts/compose.cjs exec auth node dist/database/seed.js
```

En una copia existente, empezar en `App/ms-users` y omitir la clonación. Revisar los `.env` locales después del inicializador: conserva los secretos existentes y genera los que faltan. No publicar esos archivos. STEGO recibe exclusivamente la clave pública JWT, la CA pública y su credencial de servicio, nunca el `.env` completo de AUTH. El script sincroniza emisor y audiencia con la configuración real de AUTH.

Los certificados locales duran 30 días. Para renovarlos, detener AUTH, respaldar el material de TLS y retirar únicamente el certificado y la clave de servidor vencidos; ejecutar de nuevo el inicializador y reiniciar los servicios. No retirar las claves JWT ni los datos. La CA también debe estar vigente; su renovación requiere distribuir la nueva CA a los clientes. No desactivar la validación TLS.

Si se ejecutaba previamente el Compose individual de `ms-users`, detenerlo antes de iniciar el conjunto para evitar puertos ocupados. El Compose de `App` usa otro proyecto y otro volumen de PostgreSQL: una base nueva requiere la siembra indicada. No ejecutar dos AUTH distintos suponiendo que comparten usuarios o resultados.

Servicio	API local	Documentación interactiva
AUTH	`https://localhost:3443/api/v1`	`https://localhost:3443/docs`
STEGO	`http://localhost:8000/api/v1/steganography`	`http://localhost:8000/docs`

El puerto HTTP 3001 de AUTH y el 8000 de STEGO se publican únicamente en loopback para desarrollo. El canal STEGO → AUTH utiliza siempre HTTPS/TLS 1.3 con CA verificada y credencial de servicio. Para clientes remotos se requiere terminar TLS también delante de STEGO; este Compose no publica un servicio HTTPS externo.

Cuentas de desarrollo: `jperez` ADMIN, `amendoza` ANALYST, `mgomez` USER, `cvega` USER+ANALYST y `lcastro` USER. La contraseña se consulta localmente en `ms-users/.env`, campo `SEED_PASSWORD`. No se incluye en este documento. La siembra no cambia contraseñas de usuarios ya existentes y está limitada a desarrollo.

Uso

1. Solicitar `POST /api/v1/auth/login` a AUTH con JSON `{"identifier":"amendoza","password":"<SEED_PASSWORD local>"}`.

2. Copiar el `accessToken` recibido. Enviar a STEGO `POST /api/v1/steganography/analyze` con `Authorization: Bearer <token>` y un formulario multipart con un único campo de archivo `file`. Usar las muestras de `docs/pruebas/muestras/`.

3. Consultar `GET /api/v1/steganography/results/<analysis_id>` con el mismo encabezado. ANALYST solo accede a sus resultados; ADMIN puede acceder a todos; USER no tiene permiso.

4. Importar la colección REST conjunta y configurar sus variables privadas. Añadir `ms-users/secrets/ca.crt` a los certificados confiables del cliente HTTPS. Usar `docs/pruebas/muestras` como carpeta de trabajo de Postman. En **Analizar imagen** se puede seleccionar un PNG y usar el JWT de ANALYST o ADMIN. **Consultar resultado** usa el ID del análisis. Seguir el guion actualizado.

La autorización se obtiene de nuevo en cada operación. Cambiar roles o cerrar sesión en AUTH afecta a la siguiente solicitud aunque el JWT todavía no haya expirado.

Operación independiente

Desde `App`:

```
node scripts/compose.cjs stop stego
node scripts/compose.cjs start stego
node scripts/compose.cjs stop auth
node scripts/compose.cjs start auth
node scripts/compose.cjs logs --tail 50 stego
node scripts/compose.cjs down
```

AUTH sigue respondiendo con STEGO detenido. STEGO conserva su API y almacenamiento si AUTH cae, pero deniega el análisis sin autorización vigente. `down` conserva los volúmenes; no añadir `--volumes` a la ejecución normal si se desea preservar los datos. Las migraciones de AUTH se ejecutan al iniciar su contenedor. STEGO crea su esquema SQLite versión 1 de forma idempotente.

Pruebas reproducibles

Con Python 3.12, desde `App`:

```
python -m venv ms-stego/.venv
ms-stego/.venv/Scripts/python.exe -m pip install -r ms-stego/requirements-dev.txt
ms-stego/.venv/Scripts/python.exe -m pytest -q ms-stego/tests -c ms-stego/pytest.ini
ms-stego/.venv/Scripts/python.exe tests/run.py
```

En Linux/macOS sustituir el ejecutable por `ms-stego/.venv/bin/python`. El ejecutor crea un nombre de proyecto, puertos y volúmenes temporales, construye las imágenes, prepara identidades y ejecuta las pruebas reales; al terminar elimina exclusivamente los recursos de ese proyecto temporal. No usa ni elimina los volúmenes normales. Necesita Docker iniciado y la configuración preparada con el inicializador. Los registros JSON de prueba se regeneran en `docs/pruebas/resultados/`.

Con Postman CLI instalado, `ms-stego/.venv/Scripts/python.exe tests/run.py --postman` repite también el ensayo de la colección y guarda su salida depurada.

Para regenerar las muestras, desde `App/ms-stego`: `.venv/Scripts/python.exe -m scripts.generate_samples`. El generador comprueba la recuperación del mensaje insertado; se incluyen

ejemplos con falso positivo y falso negativo. No es un corpus independiente de evaluación de precisión.

Las pruebas previas de AUTH se conservan en su documentación. Las pruebas del conjunto cubren T01-T12 con ambos servicios reales; mapa de evidencias. Las casillas de presentación en clase continúan pendientes.

Entregables

El mapa de cumplimiento relaciona la consigna con el código, la configuración y las evidencias. La documentación de entrega está reunida en App/docs. Cada servicio conserva su guía AUTH o guía STEGO para ejecución independiente.

7. Guion de sustentación con Postman

Esta guía usa Windows, Docker Desktop y la colección **Backend V1 AUTH + STEGO (simplificada)**. La terminal sirve para iniciar y detener los servicios. Postman sirve para enviar solicitudes y ver las respuestas. Durante el ensayo vas a alternar entre esas dos ventanas.

• Preparación

• T01–T12, uno por uno

• Anexo B, qué hace STEGO internamente

• Anexo C, cómo leer el resultado

• Todas las solicitudes de la colección

¿Cumplimos la independencia?

Sí. AUTH se construye con su propio Dockerfile y ejecuta un proceso NestJS. STEGO se construye con otro Dockerfile y ejecuta un proceso Python con Uvicorn. Compose permite seleccionar cuál construir, iniciar o detener. T01 comprueba la salud de AUTH con STEGO detenido. T08 comprueba la salud de STEGO con AUTH detenido y demuestra que el análisis se rechaza. Dockerfile AUTH · Dockerfile STEGO · Compose · Evidencia automatizada

PostgreSQL es infraestructura de AUTH. STEGO puede arrancar y responder en `GET /health` por su cuenta. Para analizar necesita contactar con un AUTH configurado y obtener permiso. Esa dependencia de autorización es precisamente el control que pide la consigna.

Preparación

Abre la terminal y levanta los servicios

1. Pulsa la tecla **Windows**, busca **Docker Desktop**, ábrelo y espera a que el motor esté en funcionamiento.

2. Pulsa **Windows**, escribe **PowerShell** y abre el resultado. Pega esta línea y pulsa **Enter**. Si el repositorio está en otra carpeta, usa la ruta de su carpeta `App`.

```
cd "C:\Users\ariel\Desktop\UNI\Software\09-02 2627\SSeguro\App"
```

3. Pega el comando siguiente y pulsa **Enter**. La primera construcción puede demorar. Espera a que vuelva a aparecer el indicador `PS ...>`.

```
node scripts/compose.cjs up -d --build --wait
```

4. Ejecuta `node scripts/compose.cjs ps`. Busca las filas `auth`, `stego` y `postgres` en ejecución. Si estás usando por primera vez la base del Compose conjunto, ejecuta también `node scripts/compose.cjs exec auth node dist/database/seed.js` para crear las cuentas de prueba. Si ya iniciaste sesión en Postman con esa misma base, las cuentas ya están preparadas.

5. Deja PowerShell abierto. Los demás comandos de esta guía se pegan en esa misma ventana.

Si partes de una copia nueva y faltan la configuración o las claves, sigue los pasos de preparación del README de App antes de volver al paso 3.

Prepara Postman

1. Abre **Postman Desktop**. Pulsa **Import** y selecciona backend-v1.postman_collection.json. Usa la colección que dice **(simplificada)** y comprueba que la carpeta STEGO tenga solo **Analizar imagen** y **Consultar resultado**. En la vista local de Postman con Git, se actualizó también la copia YAML. Importación en Postman

2. Si ya obtuviste `200 OK` en **Health** sin advertencia de certificado, puedes seguir. Si no, abre **Settings** → **App settings** → **Certificates**, activa **CA Certificates** y elige `App/ms-users/secrets/ca.crt`. Deja activa la verificación TLS. Certificados en Postman

3. En **Settings** → **General** → **Working directory**, selecciona la carpeta `App/docs/pruebas/muestras`. Es la **carpeta** que contiene los PNG. Carpeta de trabajo en Postman

4. En la parte superior derecha, si hay un entorno activo que no usas para esta colección, abre el selector y elige **No environment** o la opción equivalente sin entorno. Así la colección usa sus propios valores de `base_url`, `stego_url` y los tokens que guarda el login. Un entorno con una variable del mismo nombre podría usar otro valor. Prioridad de variables

5. En la barra izquierda, pulsa el título **Backend V1 AUTH + STEGO (simplificada)** y luego **Variables**. En `seed_password`, pega el valor de `SEED_PASSWORD` de tu `App/ms-users/.env` local. Para encontrarlo, abre ese archivo en el editor y busca `SEED_PASSWORD=`. Copia solo lo que está a la derecha del signo igual. En `new_user_password`, pon una contraseña temporal de al menos 12 caracteres para las cuentas de prueba. Mantén esos valores locales. Pulsa **Save** si Postman muestra ese botón. Variables de colección

6. Comprueba que `base_url` sea `https://localhost:3443/api/v1`, `internal_url` sea `https://localhost:3443` y `stego_url` sea `http://localhost:8000`, si usas los puertos predeterminados.

Aprende los cuatro movimientos que vas a repetir

Si la guía dice…	Haz esto
Abre una solicitud	En la columna izquierda, despliega la colección y la carpeta **AUTH** o **STEGO** con la flecha. Pulsa el nombre exacto de la solicitud. Se abre en el panel de la derecha.
Cambia el cuerpo JSON	En el panel de la solicitud, pulsa **Body** → **raw**. El formato debe ser **JSON**. Pega el texto en el cuadro de edición.
Adjunta un PNG	Pulsa **Body** → **form-data**. En la fila `file`, selecciona **File** junto al nombre. En **Value**, quita el archivo anterior si aparece y selecciona el nuevo desde tu equipo. Debe verse el nombre del PNG. El archivo predeterminado es `original_control.png`, relativo a la carpeta de trabajo. Postman genera el encabezado multipart.
Envía y mira el resultado	Pulsa **Send** arriba a la derecha. Abajo mira primero el código, por ejemplo `200 OK`. En el **Body de la respuesta**, más abajo, están los datos que devolvió el servidor. En sus **Headers** está `X-Request-Id`.

Las palabras entre llaves, como `{{analyst_token}}`, son espacios que Postman rellena con un valor. Si digo «guarda un valor en `user_id`», pulsa el título de la colección → **Variables** → fila `user_id`, pega el UUID sin comillas y vuelve a la solicitud. Un **UUID** es el texto largo con guiones que identifica un usuario, un rol, una asignación o un análisis. Usa siempre el ID del objeto que pide el paso.

Usa las mismas dos solicitudes para todos los casos de STEGO

1. Para obtener el JWT de ADMIN, usa **AUTH** → **Login ADMIN**. Para otra cuenta, abre **AUTH** → **Login de prueba** → **Body** → **raw** → **JSON**. Cambia únicamente `identifier` y pulsa **Send**. Para una cuenta de seed, deja `"password": "{{seed_password}}"`. Para la cuenta que creaste en el ensayo, usa `{{new_user_password}}`.

2. Abre **STEGO** → **Analizar imagen** o **Consultar resultado** y pulsa **Headers**. Edita el **Value** de la fila `Authorization`. Deja una sola fila activa con ese nombre. Para el caso sin JWT, desmarca esa fila y usa **Authorization** → **No Auth** si hay autenticación heredada. Marca de nuevo la fila al volver a los casos con token.

3. Para analizar con permiso, en **Body** → **form-data** selecciona un PNG en `file`. Para comprobar solo `401` o `403`, puedes elegir **Body** → **none**. Si Postman muestra `EISDIR`, la solicitud todavía no llegó a la API. Comprueba la carpeta de trabajo o vuelve a elegir el archivo en la fila `file`.

4. `Analizar imagen` guarda `analysis_id` solo al recibir `200`. `Consultar resultado` usa ese ID en la URL y no lleva archivo. Puedes fijar otro ID desde **Variables**.

Cuenta	Valor de Authorization que usas
`amendoza` (ANALYST)	`Bearer {{analyst_token}}`
`jperez` (ADMIN)	`Bearer {{access_token}}`
`mgomez` (USER)	`Bearer {{user_token}}`
`cvega` (otro ANALYST)	`Bearer {{other_analyst_token}}`
Cuenta nueva de prueba	`Bearer {{demo_token}}`

Los ejemplos de respuesta están debajo de cada solicitud, tanto en el JSON importable como en la copia YAML. Sirven para ver el cuerpo esperado sin enviar una solicitud. Para demostrar el parcial, usa **Send** y muestra el código y cuerpo reales. Contratos y tipos de respuesta

Los códigos de esta demo significan lo siguiente. `200` es éxito, `201` es creación y `204` es éxito sin cuerpo. `401` indica que no se pudo autenticar la solicitud y `403` indica falta de permiso. `409` indica conflicto, por ejemplo una asignación duplicada. `429` indica demasiados intentos. El login admite cinco intentos por minuto por origen, así que si ves `429`, espera un minuto antes de reintentar. El JWT dura **15 minutos** en tu configuración local.

T01. Independencia de los servicios

1. **Postman.** Pulsa **AUTH** → **Health** → **Send**. Espera `200` y `"service": "user-service"`.

2. Abre una pestaña nueva con **+**. Elige **GET** y pega `http://localhost:8000/health` en la URL. Pulsa **Send**. Espera `200` y `"service": "stego"`. Deja esa pestaña abierta. Es una comprobación técnica y se hace fuera de las dos solicitudes de STEGO de la colección.

3. **PowerShell.** Pega `node scripts/compose.cjs stop stego` y pulsa **Enter**.

4. **Postman.** Repite **Health** de AUTH. Debe seguir en `200`. La pestaña de salud de STEGO no podrá conectarse mientras su contenedor esté detenido.

5. **PowerShell.** Pega `node scripts/compose.cjs up -d --build --wait stego`. Regresa a la pestaña de salud de STEGO y confirma `200`.

Qué decir. «Cada servicio tiene su contenedor, su proceso y su API. Detuve STEGO y AUTH siguió respondiendo». En T08 comprobarás el otro sentido. El comando equivalente para desplegar solo AUTH es `node scripts/compose.cjs up -d --build --wait auth`. Ese comando arranca su PostgreSQL si hace falta.

T02. Contraseña correcta e incorrecta

1. **Postman → AUTH → Login de prueba → Body.** Verás `amendoza` en `identifier` y `{{seed_password}}` en `password`. Pulsa **Send**.

2. Confirma `200`, `expiresIn: 900` y `user.roles` con `ANALYST`. La colección guardó `accessToken` en `analyst_token`. Guarda el `user.id` de `amendoza` en una nota local como **ID de amendoza**. Ese ID sí se puede mostrar.

3. Para la contraseña incorrecta, abre una solicitud nueva con **+** arriba. Cambia el método a **POST**, pega `https://localhost:3443/api/v1/auth/login` en la barra de URL y en **Body** → **raw** → **JSON** pega:

```
{
  "identifier": "amendoza",
  "password": "incorrecta-para-la-prueba"
}
```

4. Pulsa **Send**. Espera `401`. Regresa a la pestaña de la colección para continuar.

Qué decir. «AUTH verifica la contraseña. Con las credenciales válidas devuelve un JWT con 900 segundos de vigencia».

T03. Sin JWT, con JWT alterado y con JWT vencido

Prepara el caso vencido antes de clase. Ejecuta **Login de prueba** con `amendoza` al menos 16 minutos antes. En **Variables**, copia localmente ese JWT a una nueva fila `token_vencido`. Usa el mismo login para renovar `analyst_token` al empezar. El token vencido se usa tal cual, sin editar su contenido. Mantén los JWT fuera de las capturas.

1. Abre **STEGO → Analizar imagen**. Adjunta `original_control.png`. Desmarca la fila `Authorization` en **Headers** y envía. Espera `401`.

2. Marca de nuevo la fila y escribe `Bearer token-invalido`. Envía. Espera `401`. Para mostrar una firma alterada, copia localmente un JWT válido, cambia el primer carácter **después del segundo punto** por otra letra y pon esa copia tras `Bearer` en la fila. Vuelve a enviar y espera `401`. Conserva intacta la variable `analyst_token`.

3. Cambia el valor de esa misma fila a `Bearer {{token_vencido}}`. Envía. Debe dar `401`.

4. Al terminar, restaura `Bearer {{analyst_token}}`. Si Postman no puede leer el archivo, puedes repetir estos rechazos con **Body → none**.

Qué decir. «La API rechaza la operación si falta el token, se alteró o caducó».

T04. Usuario autenticado sin permiso

1. Abre **AUTH → Login de prueba → Body**. Pon `"identifier": "mgomez"` y deja `"password": "{{seed_password}}"`. Pulsa **Send**. El `200` guarda el JWT en `user_token`.

2. Abre **STEGO → Analizar imagen → Headers**. Pon `Bearer {{user_token}}` en la fila `Authorization`. Adjunta `original_control.png` o usa **Body → none** para comprobar solo el permiso.

3. Pulsa **Send**. Debe devolver `403`. Después restaura `Bearer {{analyst_token}}`.

Qué decir. «El JWT identifica a mgomez, pero USER carece del permiso de análisis».

T05. Crear cuentas y mostrar la relación de muchos a muchos

Consigue el token de administración y los dos ID de rol

1. Abre **AUTH → Login ADMIN → Send**. Debe dar `200`. Se guarda el JWT de `jperez` en `access_token`. Las solicitudes administrativas ya llevan ese token.

2. Abre **Listar roles → Send**. En la respuesta, busca el elemento con `"rol": "USER"` y apunta el valor de su `id` como **ID de USER**. Haz lo mismo con `"rol": "ANALYST"` y guárdalo como **ID de ANALYST**.

Crea dos usuarios para asignar el mismo rol durante el ensayo

1. Abre **Crear usuario → Body → raw → JSON**. Reemplaza el ejemplo por este y pulsa **Send**:

```
{
  "cedula": "1799999001",
  "nombres": "Cuenta",
  "apellidos": "Demostracion",
  "fechaNacimiento": "1998-04-18",
  "username": "demoparcial",
  "email": "demoparcial@example.com",
  "telefono": "0990009001",
  "password": "{{new_user_password}}",
  "estado": true
```

```
}
```

2. Debe dar `201`. Copia el `id` de la respuesta y apúntalo como **ID de la cuenta A**. La contraseña se obtiene de la variable que configuraste en la preparación.

3. En el mismo JSON, cambia `cedula` por `1799999002`, `username` por `demoparcialdos`, `email` por `demoparcialdos@example.com` y `telefono` por `0990009002`. Pulsa **Send**. Copia el nuevo `id` y apúntalo como **ID de la cuenta B**. Si repites el ensayo y recibes `409`, usa cédula, teléfono, correo y nombre de usuario distintos para las cuentas nuevas.

Haz tres asignaciones y compruébalas

1. Ve a **colección → Variables**. En `user_id` pega el **ID de A** y en `role_id` el **ID de USER**. Abre **AUTH → Asignar rol → Send**. Espera `201`.

2. Conserva `user_id` de A. Cambia `role_id` al **ID de ANALYST**. Repite **Asignar rol**. Espera `201`.

3. Conserva `role_id` de ANALYST. Cambia `user_id` al **ID de B**. Repite **Asignar rol**. Espera `201`.

4. Con `user_id` de B, abre **Consultar asignaciones → Send**. Encuentra el `roleId` de ANALYST.

5. Cambia `user_id` a A y repite **Consultar asignaciones**. Debes ver dos filas, una para USER y otra con el **mismo** `roleId` **de ANALYST** que viste en B.

6. Si deseas iniciar sesión con la cuenta A, usa **Login de prueba**. Pon su nombre en `identifier` y `{{new_user_password}}` en `password`. El JWT se guarda en `demo_token`. Para los demás escenarios, vuelve a usar `amendoza` y `{{seed_password}}`.

Qué decir. «A tiene dos roles. ANALYST aparece en A y B con el mismo ID. Las asignaciones se consultan desde la API después de guardarlas».

T06. Intentar duplicar una asignación

1. Pon `user_id` de A y `role_id` de ANALYST en **Variables**.

2. Abre **Asignar rol → Send** otra vez. Espera `409`.

3. Abre **Consultar asignaciones → Send**. Comprueba que A sigue teniendo dos filas y ANALYST aparece una sola vez.

T07. Retirar el rol con el mismo JWT

1. Ejecuta **Login de prueba** con `amendoza` y `{{seed_password}}`. Abre **Analizar imagen** y revisa que **Headers** tenga `Bearer {{analyst_token}}`. Adjunta `original_control.png` y pulsa **Send**. Confirma `200`.

2. En **Variables**, pon en `user_id` el **ID de amendoza** del login. Abre **AUTH → Consultar asignaciones → Send**. Si la consulta da `401`, renueva solo **Login ADMIN**. El token de ADMIN es independiente del de ANALYST.

3. Encuentra la fila cuyo `roleId` coincide con el **ID de ANALYST**. Copia el campo `id` **de esa fila** y pégalo en `assignment_id`. Esa fila tiene su propio ID, aparte de `userId` y `roleId`.

4. Abre **Retirar asignación por id → Send**. Espera `204` y un cuerpo vacío.

5. Vuelve a **Analizar imagen** y pulsa **Send** con el mismo PNG y el mismo `{{analyst_token}}` del paso 1. Debe dar `403`. No vuelvas a iniciar sesión como `amendoza` entre los pasos 1 y 5.

6. Restaura el permiso antes de seguir. Pon `user_id` de amendoza y `role_id` de ANALYST. Pulsa **Asignar rol → Send**. Espera `201`. Vuelve a **Analizar imagen** y confirma `200` mientras el JWT siga vigente.

Qué decir. «Volví a enviar el mismo JWT. El resultado pasó de 200 a 403 porque AUTH volvió a leer las asignaciones de la base».

T08. Dejar AUTH inaccesible

1. Confirma que restauraste ANALYST y que tienes un JWT fresco. En **Analizar imagen**, prueba un `200` antes de detener nada.

2. **PowerShell.** Ejecuta `node scripts/compose.cjs stop auth`.

3. **Postman.** Vuelve a la pestaña **GET http://localhost:8000/health** del T01. Debe seguir en `200`. Luego abre **Analizar imagen** y pulsa **Send** con el mismo PNG y el JWT de ANALYST vigente.

4. Espera `503`. El cuerpo no debe traer `analysis_id`. En **Headers de la respuesta**, busca `X-Request-Id` y apúntalo como **ID de la caída** para T12.

5. **PowerShell.** Ejecuta `node scripts/compose.cjs start auth --wait`. Si el comando termina, vuelve a Postman y comprueba **Health** de AUTH. Ya puedes repetir **Analizar imagen** para recuperar un `200`.

Qué decir. «STEGO sigue vivo y responde a su propia API. Para analizar necesita una autorización verificable y, si AUTH no responde, deniega el análisis».

T09. Analizar la imagen de referencia y la imagen con carga

1. Abre **Analizar imagen** → **Body** → **form-data**. Selecciona `original_control.png` en `file`. Pulsa **Send** y busca `status: sin_indicios`.

2. Quita el archivo de la misma fila, elige `oculta_lsb.png` y pulsa **Send**. Busca `200`, `status: sospechoso` y `action: cuarentena`.

3. Anota el `analysis_id` del segundo resultado y su `X-Request-Id` en **Headers de la respuesta**. Postman ya actualizó la variable `analysis_id`. Usarás ese ID para T11 y el de solicitud para T12.

4. Muestra `sha256`, `technique`, `indicators` y `limitations`. Usa la explicación del anexo C.

5. Para probar también el permiso de **ADMIN**, ejecuta **Login ADMIN** y usa `Bearer {{access_token}}` en **Analizar imagen**. Con el mismo PNG debe dar `200`. Esa llamada actualizará `analysis_id`. Conserva aparte el ID del análisis de `amendoza` para T11 y restaura el encabezado de ANALYST.

El control es una imagen sintética con valores pares. La muestra LSB contiene texto real insertado en los bits bajos y relleno de carga alta. El generador comprueba que se puede recuperar el texto. El detector mide el balance estadístico. La muestra de ruido da un falso positivo y la de carga pequeña un falso negativo. Generador · Muestras y resultados

T10. Archivo inválido, corrupto y excesivo

1. **PowerShell.** Desde `App`, pega el bloque completo. Pulsa **Enter**. La última línea muestra una carpeta temporal con tres archivos.

```
$carpeta = Join-Path $env:TEMP ("stego-t10-" + [guid]::NewGuid())
New-Item -ItemType Directory -Path $carpeta | Out-Null
$png = (Resolve-Path 'docs/pruebas/muestras/original_control.png').Path
$bytes = [IO.File]::ReadAllBytes($png)
Copy-Item -LiteralPath $png -Destination (Join-Path $carpeta 'extension-invalida.jpg')
[IO.File]::WriteAllBytes((Join-Path $carpeta 'truncado.png'), [byte[]]($bytes[0..39]))
[IO.File]::WriteAllBytes((Join-Path $carpeta 'excesivo.png'), [byte[]]::new(5242881))
$carpeta
```

2. **Postman.** En **Settings** → **General** → **Working directory**, elige temporalmente esa carpeta. Asegúrate de que el JWT de ANALYST siga vigente. En **Analizar imagen** → **Body** → **form-data**, selecciona y envía **un archivo cada vez**.

Archivo	Código esperado	Por qué
`extension-invalida.jpg`	`415`	La extensión o el tipo enviado es incompatible con la API de PNG.
`truncado.png`	`400`	Conserva el inicio de un PNG y faltan los datos para decodificarlo. Puede quedar un registro con estado `error`.
`excesivo.png`	`413`	Tiene un byte más que el límite predeterminado de 5 MiB.

3. Mira el **código HTTP de la respuesta** y el campo `detail`. Vuelve a poner `App/docs/pruebas/muestras` como carpeta de trabajo al terminar.

T11. Consultar un resultado propio y otro ajeno

1. En **Variables** pega en `analysis_id` el ID del resultado de `amendoza` que anotaste en T09. Usa ese mismo ID durante los pasos siguientes.

2. Abre **Consultar resultado** → **Headers**. Pon `Bearer {{analyst_token}}` y pulsa **Send**. Como propietario, debe dar `200`. Esta solicitud no lleva archivo.

3. Ejecuta **Login de prueba** con `cvega` y `{{seed_password}}`. Se guarda su JWT en `other_analyst_token`. Vuelve a **Consultar resultado** y cambia solo el valor del encabezado a `Bearer {{other_analyst_token}}`. Pulsa **Send**. Debe dar `403`.

4. Ejecuta **Login ADMIN** si su sesión caducó. Vuelve a **Consultar resultado** y pon `Bearer {{access_token}}`. El mismo ID debe dar `200`. Al terminar, deja de nuevo el encabezado de ANALYST.

Qué decir. «Los dos analistas pueden analizar. Para consultar un resultado, además se verifica el propietario. ADMIN puede consultar resultados ajenos».

T12. Revisar la auditoría sin mostrar secretos

1. **Postman.** Ten a mano el **ID de la caída** de T08 y el `X-Request-Id` del análisis de T09. Puedes buscarlos en **Headers** de cada respuesta. Si se han perdido entre muchas solicitudes, repite esas dos pruebas.

2. **PowerShell.** Desde `App`, pega esta línea y pulsa **Enter**. Es una consulta de lectura a las últimas 30 filas de STEGO.

```
   node scripts/compose.cjs exec -T stego python -c "import sqlite3;
db=sqlite3.connect('/data/stego.sqlite3'); print(*db.execute('SELECT request_id, route,
status_code, analysis_id FROM audit_events ORDER BY id DESC LIMIT 30'), sep=chr(10))"
```

3. Busca la fila del análisis `200` con un `analysis_id` y la fila de la caída `503` sin `analysis_id`. En la salida de Python, `None` representa un dato vacío. Señala cómo el `request_id` de cada fila coincide con el encabezado que viste en Postman.

4. Para mostrar también AUTH en vivo, abre el `.env` local de AUTH y lee solo los valores de `DB_USERNAME` y `DB_DATABASE`. Reemplaza `USUARIO_DB` y `NOMBRE_DB` en la línea de abajo por esos dos nombres. Ejecuta la línea desde `App`. Busca el `request_id` del análisis exitoso y las decisiones de STEGO.

```
   node scripts/compose.cjs exec -T postgres psql -U USUARIO_DB -d NOMBRE_DB -c "SELECT
request_id, service_id, action, status_code, allowed FROM auth_audit_events ORDER BY
occurred_at DESC LIMIT 30"
```

Qué decir. «Podemos seguir una solicitud por su identificador y ver la acción y el estado. Las bitácoras muestran los campos de auditoría sin incluir contraseñas, claves ni JWT completos». Ejemplo depurado de AUTH · Ejemplo de STEGO

Al terminar el ensayo puedes detener el conjunto con `node scripts/compose.cjs down`. Ese comando conserva los datos de los volúmenes.

Anexo B. La decisión interna

1. Haz clic en **Login de prueba**, deja `amendoza` en el cuerpo y envía. Postman habla con AUTH en `https://localhost:3443/api/v1/auth/login` y guarda el JWT.

2. Haz clic en **Analizar imagen** y envía el PNG. Postman llama a STEGO en `http://localhost:8000/api/v1/steganography/analyze` con el JWT en **Headers** → **Authorization**.

3. STEGO verifica la firma y vigencia del JWT. Luego llama a **AUTH** desde su red interna por HTTPS. La ruta exacta es `POST /internal/authorization/check`. El encabezado `X-Service-Token` identifica a STEGO como servicio y la CA permite verificar el certificado de AUTH.

4. STEGO envía `subject_token` (JWT del usuario), `action: "steganography.analyze"` y `resource_id: null` porque todavía no hay análisis. AUTH valida el estado del usuario y lee sus asignaciones vigentes. Devuelve `subject_id`, `action`, `resource_id`, `allowed` y `reason`.

5. Una respuesta interna `200` puede incluir `allowed: false`. En ese caso el cliente recibe `403` de STEGO. Cuando AUTH está caído, STEGO devuelve `503`. Si la respuesta interna no se puede verificar, STEGO puede devolver `502`. En todos esos casos se deniega el análisis.

6. En una consulta de resultado, la acción es `steganography.results.read`. STEGO envía el ID del análisis y el propietario leído de SQLite. Cada petición pide una decisión nueva. Esto explica por qué T07 funciona con el mismo JWT.

En la demo, muestra la llamada interna con la auditoría de T12. Cuando **Analizar imagen** devuelve `200`, usa el mismo `X-Request-Id` en los registros de AUTH y STEGO. En AUTH aparece la acción `steganography.analyze` y la decisión. En STEGO aparece el análisis que se creó. La credencial de servicio se usa internamente. Contrato OpenAPI de AUTH

Anexo C. El resultado del análisis

Abre **Analizar imagen**, adjunta `oculta_lsb.png` y pulsa **Send**. Lee el JSON de la respuesta **de abajo**, no el cuerpo de la solicitud de arriba.

Campo	Qué puedes decir
`analysis_id`	«Es el identificador del análisis. Con él consulto el resultado después».
`user_id`	«Es el UUID de quien solicitó el análisis. AUTH lo usa para evaluar quién puede consultar este resultado».
`filename`, `sha256`	«Son el nombre recibido y la huella de los bytes de este archivo».
`technique`, `indicators`	«Aquí está el método y los números que calculó sobre los píxeles».
`created_at`, `status`, `action`, `limitations`	«Aquí consta cuándo ocurrió, cómo se clasificó, qué política aplicó y qué limitaciones tiene el resultado».

En las muestras guardadas, la de carga alta tiene `chi_square ≈ 125.30` y `equiprobability_p_value ≈ 0.5511`. Con la regla del proyecto, ese patrón de pares balanceados se marca `sospechoso` y se guarda en cuarentena. El número mide compatibilidad con ese patrón estadístico. No es una probabilidad de que la imagen contenga

un mensaje. El `0.82` del PDF es un ejemplo de estructura. Para la demo usa los valores que veas en tu respuesta. Método · Contrato OpenAPI de STEGO

Para qué sirve cada solicitud

Un **endpoint** es una combinación de método y ruta, como `POST /api/v1/auth/login`. Una **solicitud guardada** de Postman puede usar ese mismo endpoint con otra cuenta, otro archivo o un token distinto. Por ejemplo, **Login ADMIN** y **Login de prueba** llaman a la misma ruta. Al lado de cada nombre, `GET` suele leer, `POST` envía datos o realiza una acción, `PATCH` cambia campos y `DELETE` retira un recurso o asignación.

Carpeta AUTH

En esta tabla, las rutas abreviadas que empiezan con `/auth`, `/users`, `/roles` o `/user-roles` van después de `https://localhost:3443/api/v1`. La ruta interna va directamente después de `https://localhost:3443`.

Solicitud	Método y ruta	Qué hace y qué usa
Health	`GET /health`	Comprueba que AUTH responde. Es pública y no requiere token.
Login ADMIN	`POST /auth/login`	Entra como `jperez` con `seed_password`. Guarda el JWT en `access_token` para administrar.
Login de prueba	`POST /auth/login`	Cambia la cuenta en **Body**. Guarda el JWT en la variable correspondiente a esa cuenta.
Perfil	`GET /auth/profile`	Devuelve la identidad y roles de la sesión del token enviado. Esta copia usa `access_token`.
Perfil alias me	`GET /auth/me`	Consulta el mismo perfil desde la ruta alternativa.
Crear usuario	`POST /users`	Como ADMIN, crea una cuenta con el JSON de **Body**. Devuelve `201` y el ID del usuario.
Listar usuarios	`GET /users`	Como ADMIN, lista usuarios en páginas. Busca los elementos en `data`.
Consultar usuario	`GET /users/{id}`	Como ADMIN, busca un usuario específico. El ID sale de `user_id`.
Consultar cédula	`GET /users/cedula/{cedula}`	Como ADMIN, busca por cédula. Cambia los últimos diez dígitos de la URL por la cédula de la cuenta de prueba.
Actualizar usuario	`PATCH /users/{id}`	Como ADMIN, modifica los campos enviados en **Body** para `user_id`. El ejemplo cambia `nombres`.

Solicitud	Método y ruta	Qué hace y qué usa
Eliminar usuario	`DELETE /users/{id}`	Como ADMIN, hace un borrado lógico de `user_id` y devuelve `204`. Si lo ensayas, usa una cuenta de prueba.
Crear rol	`POST /roles`	Como ADMIN, crea un rol de catálogo. El ejemplo usa `SUPERVISOR`. Los permisos de análisis actuales corresponden a ADMIN o ANALYST.
Listar roles	`GET /roles`	Como ADMIN, lista los roles y permite copiar sus ID. El nombre del rol aparece en `rol`.
Consultar rol	`GET /roles/{id}`	Como ADMIN, muestra el detalle del rol guardado en `role_id`.
Actualizar rol	`PATCH /roles/{id}`	Como ADMIN, cambia campos de `role_id`. El ejemplo modifica la descripción.
Eliminar rol	`DELETE /roles/{id}`	Como ADMIN, hace un borrado lógico del rol y devuelve `204`. Si lo ensayas, usa el rol de prueba.
Asignar rol	`POST /user-roles`	Como ADMIN, une `user_id` con `role_id`. Devuelve la fila creada con un ID de asignación.
Consultar asignaciones	`GET /user-roles/user/{userId}`	Como ADMIN, muestra las asignaciones de `user_id` con sus ID y rol asociado.
Retirar asignación por id	`DELETE /user-roles/{id}`	Como ADMIN, retira únicamente la fila `assignment_id`. Se usa en T07.
Logout	`POST /auth/logout`	Usa el token de `access_token`, devuelve `204` e invalida los JWT emitidos anteriormente para esa cuenta. Déjalo para el final si todavía necesitas el token de ADMIN.

Carpeta STEGO

Estas son las dos solicitudes de negocio. Cada una se reutiliza cambiando el JWT, el archivo o el ID.

Solicitud	Método y ruta	Qué hace y qué usa
Analizar imagen	`POST /api/v1/steganography/analyze`	Envía `file` y el JWT de ANALYST o ADMIN. Con `200` guarda `analysis_id`. Reutiliza esta solicitud para los rechazos.
Consultar resultado	`GET /api/v1/steganography/results/{analysis_id}`	Devuelve el análisis al propietario con permiso o a ADMIN. Puedes probar un análisis ajeno cambiando solo el token.

Anexo A. Mapa de cumplimiento

Referencia de evaluación. Proyecto Integrador del Primer Parcial, páginas 2–4 y anexos A–E.

2.1 Arquitectura e independencia

Qué se exige	Dónde se implementa y cómo se evidencia
Dos aplicaciones con procesos, configuraciones, endpoints y documentación propios	AUTH arranca desde main.ts y server.ts. STEGO tiene su aplicación en main.py y configuración en settings.py. Cada servicio tiene su propio README AUTH o README STEGO, contrato y Dockerfile.
Contenedores separados, arranque y parada autónomos	Docker Compose declara `auth`, `stego` y PostgreSQL como infraestructura. T01 y T08 demuestran que la API del servicio que sigue activo responde cuando se detiene el otro. Pruebas.
Justificar tecnologías y separar datos	NestJS/TypeScript con PostgreSQL en AUTH. FastAPI/Python con Pillow, NumPy, SciPy y SQLite en STEGO. El diagrama muestra límites de confianza, redes, procesos y almacenamiento. Arquitectura.

2.2 AUTH

Qué se exige	Implementación y evidencia
AUTH como autoridad de identidades y roles; CRUD	Usuarios, roles y asignaciones. Las operaciones de administración exigen ADMIN.
`users`, `roles`, `user_roles`, PK, FK, unicidad y auditoría	User, Role, UserRole y migración AUTH V2. El par `(user_id, role_id)` es único y las asignaciones son filas de PostgreSQL. T05–T06 verifican N:M y rechazo de duplicados.
Credenciales y JWT	AuthService valida credenciales persistidas. UsersService protege contraseñas con Argon2id. AuthModule y JwtStrategy aplican RS256, emisor, audiencia, vigencia, `sub` y versión de sesión. Gestión de claves y flujo.
RBAC actual y mínimo privilegio	JwtStrategy y UsersService consultan usuario, roles y asignaciones activos. RolesGuard aplica la política por ruta. ADMIN administra, ANALYST analiza y lee lo propio, USER usa su perfil y sesión. La política por acción está declarada en código; las asignaciones a personas y su vigencia se resuelven en PostgreSQL. T04, T07 y T11 muestran denegación, revocación y propiedad.

2.3 STEGO

Qué se exige	Implementación y evidencia
Multipart y validación de archivo	API y middleware exigen un campo `file`, extensión PNG, MIME, firma y tamaño. Trabajador comprueba IHDR, dimensiones y decodificación; procesamiento aplica el plazo y el entorno del proceso. T10 evidencia 400, 413 y 415.
Técnica real con parámetros y muestras	Detector aplica el contraste chi-cuadrado por pares LSB a PNG de 8 bits compatibles. Método, parámetros y límites, muestras, resultados reales y T09.
Clasificación, persistencia y política	`sin_indicios`, `sospechoso`, `no_concluyente` y `error`. Store guarda ID, propietario, SHA-256, técnica, indicadores, fecha, resultado y política. La API devuelve advertencia o cuarentena y rechaza entradas incompatibles. El volumen es privado y aplica retención; no hay ruta pública para descargar archivos.
Tratamiento aislado	Trabajador, Dockerfile STEGO y Compose. Procesamiento sin root y con límites de red, CPU, memoria y tiempo. La suite integrada comprueba que el trabajador no puede abrir un socket.

2.4 Comunicación segura y RBAC

Qué se exige	Implementación y evidencia
Cliente con JWT y validación local	AUTH entrega el JWT. STEGO verifica firma y claims con la clave pública y recibe el token mediante `Authorization: Bearer`. Security, T02–T03.
Decisión vigente, acción y recurso	Middleware STEGO solicita decisión antes de consumir el archivo. Para consultar un resultado, obtiene su propietario de SQLite. Endpoint interno AUTH vuelve a validar sesión y roles persistidos. T07 y T11.
Identidad de servicio, canal seguro y denegación por defecto	`ServiceAuthGuard` exige TLS y `X-Service-Token`; STEGO valida el certificado de AUTH. Security comprueba sujeto, acción, recurso y estructura de la respuesta. La caída devuelve 503 y la respuesta no verificable devuelve 502. Pruebas de seguridad y T08.
Separación y rechazo de permisos manipulados	Compose conecta solo AUTH a `auth-data` y monta material público en STEGO. STEGO carece de contraseñas de identidad y clave privada JWT. T04 prueba que las cabeceras de rol falsificadas no conceden acceso. Matriz de controles.

2.5 Contrato mínimo

Método	Ruta del proyecto	Acceso	Código
POST	`/api/v1/auth/login`	Público, con límite	AuthController
GET	`/api/v1/auth/profile`	Autenticado	AuthController
POST	`/api/v1/auth/logout`	Sesión autenticada	AuthController
POST y GET	`/api/v1/users`	ADMIN	UsersController
PATCH	`/api/v1/users/{id}`	ADMIN	UsersController
POST y GET	`/api/v1/roles`	ADMIN	RolesController
PATCH	`/api/v1/roles/{id}`	ADMIN	RolesController
POST	`/api/v1/user-roles`	ADMIN	UserRolesController
DELETE	`/api/v1/user-roles/{id}`	ADMIN	UserRolesController
POST	`/internal/authorization/check`	Cliente STEGO autenticado sobre TLS	InternalAuthorizationController
POST	`/api/v1/steganography/analyze`	ADMIN o ANALYST	API STEGO
GET	`/api/v1/steganography/results/{analysis_id}`	ANALYST propietario o ADMIN	API STEGO

La colección conserva las operaciones de administración de AUTH y agrupa STEGO en **Analizar imagen** y **Consultar resultado**. La autorización interna permanece en OpenAPI y la invoca STEGO. La salud de STEGO se verifica aparte en `GET /health`. Los códigos 401/403/413/415 se verifican en T03, T04, T10 y T11. Auditoría AUTH y auditoría STEGO registran datos acotados. OpenAPI AUTH, OpenAPI STEGO, colección Postman y resultados.

3 Entregables y presentación

Bloque	Entregable y ubicación
1. Código fuente separado y construcción	AUTH, STEGO, dependencias AUTH, dependencias STEGO, Dockerfile AUTH y Dockerfile STEGO.
2. Preparación y persistencia	README conjunto, .env.example AUTH, .env.example STEGO, migración inicial, migración AUTH V2, siembra de desarrollo y Compose. Los volúmenes separan las identidades de los resultados.
3. Diagramas	Arquitectura y modelo entidad–relación, con las dos imágenes SVG.
4. JWT, RBAC, permisos, OpenAPI y detección	Seguridad, contratos y método reúne la explicación y enlaza los contratos completos.
5. Evidencia de desarrollo seguro	Matriz de riesgos, que vincula cada riesgo con controles, pruebas y principios solicitados.
6. Pruebas, solicitudes, muestras y auditoría	Pruebas y evidencias, colección REST, muestras, auditoría AUTH y auditoría STEGO. La consigna requiere los tipos de prueba y los escenarios del anexo D. La implementación usa Jest, Node.js, pytest y HTTPX.
7. Sustentación	Guion detallado para Postman. Las pruebas automatizadas respaldan lo que el equipo debe demostrar en la sesión.

Cobertura del documento completo

Parte de la consigna	Evidencia que permite defenderla
Resumen y §§1–1.2, objetivo y responsabilidades	Arquitectura, servicios separados, identidades N:M y análisis real.
§1.3, alcance	La entrega es backend. Salas, interfaz, mensajería y E2EE pertenecen a fases posteriores. Los documentos de actividades anteriores se identifican por su propio alcance.
§§2.1–2.5	Matrices de este mapa, contratos OpenAPI y rutas del código enlazadas arriba.
§3, código y configuración	README, Dockerfiles, dependencias, `.env.example`, migraciones, seed y Compose enlazados arriba.
§3, documentos y evidencias	Diagramas SVG, explicación JWT/RBAC, matriz de permisos, método, matriz de riesgos, colección REST, muestras y registros. Índice de entregables.
§3, sustentación	Guion. La ejecución por el equipo durante la presentación se registra por separado en los checklists.
§4, rúbrica de 20 puntos	Arquitectura (3), identidad (3), JWT (3), RBAC/integración (4), detección (5) y pruebas/documentación (2). Cada grupo está enlazado con su implementación y T01–T12. La calificación corresponde al docente.
Anexo A	PK, FK, UUID propio y par único en entidades y migraciones de AUTH.
Anexo B	Endpoint interno sobre TLS, identidad de servicio, decisión actual por acción/recurso y denegación por defecto.
Anexo C	Contrato de resultados con SHA-256, indicadores calculados, fecha, clasificación, política y limitaciones.
Anexo D	Pruebas T01–T12, código ejecutable y resultados reproducibles.
Anexo E	Matriz M01–M07, que contiene los seis grupos de activos del ejemplo y el riesgo del detector.
§6, referencias	Los checklists conservan la bibliografía del PDF. La documentación técnica vincula el método, los principios ISO, JWT, Zero Trust y carga segura con sus fuentes.

El checklist 04 conserva el texto del 03 y marca la comprobación previa P09. Las casillas P01–P08 se completan cuando el equipo realice la presentación.

Anexo B. Referencia de las APIs OpenAPI

Esta referencia se genera de los JSON vigentes del repositorio. Las referencias a modelos nombrados se resuelven en los esquemas del mismo servicio. Los archivos originales completos se incluyen como adjuntos.

AUTH - AUTH Backend V1

OpenAPI 3.0.0 | Versión 2.0

Seguridad bearer: {"scheme": "bearer", "bearerFormat": "JWT", "type": "http"}

Seguridad ServiceToken: {"type": "apiKey", "in": "header", "name": "X-Service-Token"}

GET /api/v1/health

Autorización declarada: []

HTTP	Descripción / modelo
200	Operación exitosa | application/json: Health
429	Límite de solicitudes excedido | application/json: AuthError
500	Error interno | application/json: AuthError

POST /api/v1/users

Autorización declarada: [{"bearer": []}]

Cuerpo application/json: CreateUserDto

HTTP	Descripción / modelo
201	Recurso creado | application/json: User
400	Entrada o parámetro inválido | application/json: AuthError
401	Credenciales, sesión o identidad de servicio inválidas | application/json: AuthError
403	La sesión no tiene permiso | application/json: AuthError
409	Conflicto de unicidad | application/json: AuthError
413	Carga excesiva | application/json: AuthError
415	Codificación no admitida | application/json: AuthError
429	Límite de solicitudes excedido | application/json: AuthError
500	Error interno | application/json: AuthError

GET /api/v1/users

Autorización declarada: [{"bearer": []}]

Parámetro page [query]: integer; minimum=1; default=1; opcional

Parámetro limit [query]: integer; minimum=1; maximum=100; default=20; opcional

HTTP	Descripción / modelo
200	Operación exitosa | application/json: UsersPage

HTTP	Descripción / modelo
400	Entrada o parámetro inválido | application/json: AuthError
401	Credenciales, sesión o identidad de servicio inválidas | application/json: AuthError
403	La sesión no tiene permiso | application/json: AuthError
429	Límite de solicitudes excedido | application/json: AuthError
500	Error interno | application/json: AuthError

GET /api/v1/users/cedula/{cedula}

Autorización declarada: [{"bearer": []}]

Parámetro cedula [path]: string; obligatorio

HTTP	Descripción / modelo
200	Operación exitosa | application/json: User
400	Entrada o parámetro inválido | application/json: AuthError
401	Credenciales, sesión o identidad de servicio inválidas | application/json: AuthError
403	La sesión no tiene permiso | application/json: AuthError
404	Entidad no encontrada | application/json: AuthError
429	Límite de solicitudes excedido | application/json: AuthError
500	Error interno | application/json: AuthError

GET /api/v1/users/{id}

Autorización declarada: [{"bearer": []}]

Parámetro id [path]: string; obligatorio

HTTP	Descripción / modelo
200	Operación exitosa | application/json: User
400	Entrada o parámetro inválido | application/json: AuthError
401	Credenciales, sesión o identidad de servicio inválidas | application/json: AuthError
403	La sesión no tiene permiso | application/json: AuthError
404	Entidad no encontrada | application/json: AuthError
429	Límite de solicitudes excedido | application/json: AuthError
500	Error interno | application/json: AuthError

PATCH /api/v1/users/{id}

Autorización declarada: [{"bearer": []}]

Parámetro id [path]: string; obligatorio

Cuerpo application/json: UpdateUserDto

HTTP	Descripción / modelo
200	Operación exitosa | application/json: User
400	Entrada o parámetro inválido | application/json: AuthError
401	Credenciales, sesión o identidad de servicio inválidas | application/json: AuthError
403	La sesión no tiene permiso | application/json: AuthError
404	Entidad no encontrada | application/json: AuthError
409	Conflicto de unicidad | application/json: AuthError
413	Carga excesiva | application/json: AuthError
415	Codificación no admitida | application/json: AuthError
429	Límite de solicitudes excedido | application/json: AuthError
500	Error interno | application/json: AuthError

DELETE /api/v1/users/{id}

Autorización declarada: [{"bearer": []}]

Parámetro id [path]: string; obligatorio

HTTP	Descripción / modelo
204	Operación exitosa sin contenido
400	Entrada o parámetro inválido | application/json: AuthError
401	Credenciales, sesión o identidad de servicio inválidas | application/json: AuthError
403	La sesión no tiene permiso | application/json: AuthError
404	Entidad no encontrada | application/json: AuthError
429	Límite de solicitudes excedido | application/json: AuthError
500	Error interno | application/json: AuthError

POST /api/v1/roles

Autorización declarada: [{"bearer": []}]

Cuerpo application/json: CreateRoleDto

HTTP	Descripción / modelo
201	Recurso creado | application/json: Role
400	Entrada o parámetro inválido | application/json: AuthError
401	Credenciales, sesión o identidad de servicio inválidas | application/json: AuthError
403	La sesión no tiene permiso | application/json: AuthError
409	Conflicto de unicidad | application/json: AuthError

HTTP	Descripción / modelo
413	Carga excesiva | application/json: AuthError
415	Codificación no admitida | application/json: AuthError
429	Límite de solicitudes excedido | application/json: AuthError
500	Error interno | application/json: AuthError

GET /api/v1/roles

Autorización declarada: [{"bearer": []}]

Parámetro page [query]: integer; minimum=1; default=1; opcional

Parámetro limit [query]: integer; minimum=1; maximum=100; default=20; opcional

HTTP	Descripción / modelo
200	Operación exitosa | application/json: RolesPage
400	Entrada o parámetro inválido | application/json: AuthError
401	Credenciales, sesión o identidad de servicio inválidas | application/json: AuthError
403	La sesión no tiene permiso | application/json: AuthError
429	Límite de solicitudes excedido | application/json: AuthError
500	Error interno | application/json: AuthError

GET /api/v1/roles/{id}

Autorización declarada: [{"bearer": []}]

Parámetro id [path]: string; obligatorio

HTTP	Descripción / modelo
200	Operación exitosa | application/json: Role
400	Entrada o parámetro inválido | application/json: AuthError
401	Credenciales, sesión o identidad de servicio inválidas | application/json: AuthError
403	La sesión no tiene permiso | application/json: AuthError
404	Entidad no encontrada | application/json: AuthError
429	Límite de solicitudes excedido | application/json: AuthError
500	Error interno | application/json: AuthError

PATCH /api/v1/roles/{id}

Autorización declarada: [{"bearer": []}]

Parámetro id [path]: string; obligatorio

Cuerpo application/json: UpdateRoleDto

HTTP	Descripción / modelo
200	Operación exitosa | application/json: Role
400	Entrada o parámetro inválido | application/json: AuthError
401	Credenciales, sesión o identidad de servicio inválidas | application/json: AuthError
403	La sesión no tiene permiso | application/json: AuthError
404	Entidad no encontrada | application/json: AuthError
409	Conflicto de unicidad | application/json: AuthError
413	Carga excesiva | application/json: AuthError
415	Codificación no admitida | application/json: AuthError
429	Límite de solicitudes excedido | application/json: AuthError
500	Error interno | application/json: AuthError

DELETE /api/v1/roles/{id}

Autorización declarada: [{"bearer": []}]

Parámetro id [path]: string; obligatorio

HTTP	Descripción / modelo
204	Operación exitosa sin contenido
400	Entrada o parámetro inválido | application/json: AuthError
401	Credenciales, sesión o identidad de servicio inválidas | application/json: AuthError
403	La sesión no tiene permiso | application/json: AuthError
404	Entidad no encontrada | application/json: AuthError
429	Límite de solicitudes excedido | application/json: AuthError
500	Error interno | application/json: AuthError

POST /api/v1/user-roles

Autorización declarada: [{"bearer": []}]

Cuerpo application/json: CreateUserRoleDto

HTTP	Descripción / modelo
201	Recurso creado | application/json: Assignment
400	Entrada o parámetro inválido | application/json: AuthError
401	Credenciales, sesión o identidad de servicio inválidas | application/json: AuthError
403	La sesión no tiene permiso | application/json: AuthError
404	Entidad no encontrada | application/json: AuthError

HTTP	Descripción / modelo
409	Conflicto de unicidad | application/json: AuthError
413	Carga excesiva | application/json: AuthError
415	Codificación no admitida | application/json: AuthError
429	Límite de solicitudes excedido | application/json: AuthError
500	Error interno | application/json: AuthError

GET /api/v1/user-roles/user/{userId}

Autorización declarada: [{"bearer": []}]

Parámetro userId [path]: string; obligatorio

HTTP	Descripción / modelo
200	Operación exitosa | application/json: array de Assignment
400	Entrada o parámetro inválido | application/json: AuthError
401	Credenciales, sesión o identidad de servicio inválidas | application/json: AuthError
403	La sesión no tiene permiso | application/json: AuthError
404	Entidad no encontrada | application/json: AuthError
429	Límite de solicitudes excedido | application/json: AuthError
500	Error interno | application/json: AuthError

DELETE /api/v1/user-roles/{id}

Autorización declarada: [{"bearer": []}]

Parámetro id [path]: string; obligatorio

HTTP	Descripción / modelo
204	Operación exitosa sin contenido
400	Entrada o parámetro inválido | application/json: AuthError
401	Credenciales, sesión o identidad de servicio inválidas | application/json: AuthError
403	La sesión no tiene permiso | application/json: AuthError
404	Entidad no encontrada | application/json: AuthError
429	Límite de solicitudes excedido | application/json: AuthError
500	Error interno | application/json: AuthError

POST /api/v1/auth/login

Autorización declarada: []

Cuerpo application/json: LoginDto

HTTP	Descripción / modelo
200	JWT RS256 y perfil de login | application/json: object; campos: accessToken, tokenType, expiresIn, user
400	Entrada o parámetro inválido | application/json: AuthError
401	Credenciales inválidas | application/json: AuthError
413	Carga excesiva | application/json: AuthError
415	Codificación no admitida | application/json: AuthError
429	Límite de solicitudes excedido | application/json: AuthError
500	Error interno | application/json: AuthError

GET /api/v1/auth/profile

Autorización declarada: [{"bearer": []}]

HTTP	Descripción / modelo
200	Usuario activo y roles vigentes desde PostgreSQL | application/json: object; campos: userId, username, email, roles
401	Credenciales, sesión o identidad de servicio inválidas | application/json: AuthError
429	Límite de solicitudes excedido | application/json: AuthError
500	Error interno | application/json: AuthError

GET /api/v1/auth/me

Autorización declarada: [{"bearer": []}]

HTTP	Descripción / modelo
200	Usuario activo y roles vigentes desde PostgreSQL | application/json: object; campos: userId, username, email, roles
401	Credenciales, sesión o identidad de servicio inválidas | application/json: AuthError
429	Límite de solicitudes excedido | application/json: AuthError
500	Error interno | application/json: AuthError

POST /api/v1/auth/logout

Autorización declarada: [{"bearer": []}]

HTTP	Descripción / modelo
204	Operación exitosa sin contenido
401	Credenciales, sesión o identidad de servicio inválidas | application/json: AuthError
413	Carga excesiva | application/json: AuthError
415	Codificación no admitida | application/json: AuthError
429	Límite de solicitudes excedido | application/json: AuthError

HTTP	Descripción / modelo
500	Error interno | application/json: AuthError

POST /internal/authorization/check

Decisión vigente para STEGO sobre TLS; resource_owner_id procede del registro del análisis en STEGO

Autorización declarada: [{"ServiceToken": []}]

Cuerpo application/json: CheckAuthorizationDto

HTTP	Descripción / modelo
200	Decisión actual, allowed true o false; no constituye permiso permanente | application/json: object; campos: subject_id, action, resource_id, allowed, reason
400	Entrada o parámetro inválido | application/json: AuthError
401	Servicio, canal o token de usuario inválido | application/json: AuthError
413	Carga excesiva | application/json: AuthError
415	Codificación no admitida | application/json: AuthError
429	Límite de solicitudes excedido | application/json: AuthError
500	Error interno | application/json: AuthError

Modelos de datos de AUTH

CreateUserDto

Campo	Tipo y restricciones	Requerido
cedula	string; pattern=^[0-9]{10}$	Sí
nombres	string; minLength=2; maxLength=100	Sí
apellidos	string; minLength=2; maxLength=100	Sí
fechaNacimiento	string; pattern=^\d{4}-\d{2}-\d{2}$	Sí
username	string; minLength=4; maxLength=50; pattern=^[a-z0-9._-]+$	Sí
email	string (email); maxLength=150	Sí
telefono	string; pattern=^[0-9]{10}$	Sí
password	string; minLength=12; maxLength=128	Sí
estado	boolean	No

User

Campo	Tipo y restricciones	Requerido
id	string (uuid)	No
cedula	string	No
nombres	string	No
apellidos	string	No
username	string	No
email	string (email)	No
telefono	string	No
fechaNacimiento	string (date)	No
estado	boolean	No
tokenVersion	integer	No
version	integer	No
fechaCreacion	string (date-time)	No
fechaActualizacion	string (date-time)	No
fechaEliminacion	string (date-time); nullable=True	No
userRoles	array de Assignment	No

Role

Campo	Tipo y restricciones	Requerido
id	string (uuid)	No
rol	string	No
description	string	No
estado	boolean	No
version	integer	No
fechaCreacion	string (date-time)	No
fechaActualizacion	string (date-time)	No
fechaEliminacion	string (date-time); nullable=True	No

UserRole

Campo	Tipo y restricciones	Requerido
id	string	Sí
userId	string	Sí
roleId	string	Sí
user	User	Sí
role	Role	Sí
fechaAsignacion	string (date-time)	Sí
estado	boolean	Sí
version	number	Sí
fechaActualizacion	string (date-time)	Sí

UpdateUserDto

Campo	Tipo y restricciones	Requerido
nombres	string; minLength=2; maxLength=100	No
apellidos	string; minLength=2; maxLength=100	No
fechaNacimiento	string; pattern=^\d{4}-\d{2}-\d{2}$	No
username	string; minLength=4; maxLength=50; pattern=^[a-z0-9._-]+$	No
email	string (email); maxLength=150	No
telefono	string; pattern=^[0-9]{10}$	No
password	string; minLength=12; maxLength=128	No
estado	boolean	No

CreateRoleDto

Campo	Tipo y restricciones	Requerido
rol	string; minLength=2; maxLength=50; pattern=^[A-Z0-9_]+$	Sí
description	string; minLength=2; maxLength=500	Sí
estado	boolean	No

UpdateRoleDto

Campo	Tipo y restricciones	Requerido
rol	string; minLength=2; maxLength=50; pattern=^[A-Z0-9_]+$	No
description	string; minLength=2; maxLength=500	No
estado	boolean	No

CreateUserRoleDto

Campo	Tipo y restricciones	Requerido
userId	string (uuid)	Sí
roleId	string (uuid)	Sí

LoginDto

Campo	Tipo y restricciones	Requerido
identifier	string; minLength=4; maxLength=150	Sí
password	string; minLength=1; maxLength=128	Sí

CheckAuthorizationDto

Para results.read ambos UUID son obligatorios. En analyze pueden omitirse.

Campo	Tipo y restricciones	Requerido
subject_token	string; maxLength=8192	Sí
action	string; enum=['steganography.analyze', 'steganography.results.read']	Sí
resource_id	string (uuid); nullable=True	No
resource_owner_id	string (uuid); nullable=True	No

AuthError

Campo	Tipo y restricciones	Requerido
statusCode	integer	Sí
message	string / array de string	Sí
timestamp	string (date-time)	Sí
path	string	Sí
requestId	string	No

Health

Campo	Tipo y restricciones	Requerido
status	string	No
service	string	No
timestamp	string (date-time)	No

Assignment

Campo	Tipo y restricciones	Requerido
id	string (uuid)	No
userId	string (uuid)	No
roleId	string (uuid)	No
fechaAsignacion	string (date-time)	No
estado	boolean	No
version	integer	No
fechaActualizacion	string (date-time)	No
role	Role	No

UsersPage

Campo	Tipo y restricciones	Requerido
data	array de User	No
meta	object; campos: page, limit, total, totalPages	No

RolesPage

Campo	Tipo y restricciones	Requerido
data	array de Role	No
meta	object; campos: page, limit, total, totalPages	No

STEGO - STEGO Backend V1

OpenAPI 3.1.0 | Versión 1.0.0

Seguridad BearerAuth: {"type": "http", "scheme": "bearer", "bearerFormat": "JWT"}

GET /health

Health

Autorización declarada: []

HTTP	Descripción / modelo
200	Successful Response | application/json: ServiceHealth

POST /api/v1/steganography/analyze

Upload

Autorización declarada: [{"BearerAuth": []}]

Cuerpo multipart/form-data: object; campos: file

file: string (binary)

HTTP	Descripción / modelo
200	Successful Response | application/json: AnalysisResult

HTTP	Descripción / modelo
401	JWT o sesión inválidos | application/json: ApiError
403	No tiene permiso sobre la operación o el recurso | application/json: ApiError
502	AUTH devolvió una decisión no verificable | application/json: ApiError
503	AUTH, capacidad o procesamiento no disponibles | application/json: ApiError / RecordedAnalysisError
500	Error interno | application/json: ApiError
400	Formulario inválido o PNG corrupto | application/json: ApiError / RecordedAnalysisError
413	Carga o dimensiones excesivas | application/json: ApiError / RecordedAnalysisError
415	Formato no admitido | application/json: ApiError / RecordedAnalysisError

GET /api/v1/steganography/results/{analysis_id}

Result

Autorización declarada: [{"BearerAuth": []}]

Parámetro analysis_id [path]: string; obligatorio

HTTP	Descripción / modelo
200	Successful Response | application/json: AnalysisResult
401	JWT o sesión inválidos | application/json: ApiError
403	No tiene permiso sobre la operación o el recurso | application/json: ApiError
502	AUTH devolvió una decisión no verificable | application/json: ApiError
503	AUTH no pudo proporcionar una decisión | application/json: ApiError
500	Error interno | application/json: ApiError
400	Identificador de análisis inválido | application/json: ApiError
404	Análisis no encontrado | application/json: ApiError

Modelos de datos de STEGO

AnalysisResult

Campo	Tipo y restricciones	Requerido
analysis_id	string	Sí
user_id	string	Sí
filename	string	Sí
sha256	string	Sí
technique	string	Sí
indicators	object	Sí
created_at	string	Sí
status	string	Sí
action	string	Sí
limitations	string	Sí

ApiError

Campo	Tipo y restricciones	Requerido
detail	string	Sí

RecordedAnalysisError

Campo	Tipo y restricciones	Requerido
analysis_id	string	Sí
user_id	string	Sí
filename	string	Sí
sha256	string	Sí
technique	string	Sí
indicators	object	Sí
created_at	string	Sí
status	string	Sí
action	string	Sí
limitations	string	Sí
detail	string	Sí

ServiceHealth

status: string; service: string. Campos requeridos: status, service

Anexo C. Inventario documental y archivos adjuntos

Archivo de origen	Tamaño
App/docs/README.md	1,814 bytes
App/docs/00_mapa_cumplimiento.md	12,509 bytes
App/docs/01_arquitectura_y_modelo_datos.md	2,179 bytes
App/docs/02_seguridad_contratos_y_metodo.md	14,574 bytes
App/docs/03_matriz_riesgos.md	8,433 bytes
App/docs/04_pruebas_y_evidencias.md	6,891 bytes
App/docs/05_guion_sustentacion_postman.md	31,077 bytes
App/README.md	7,047 bytes
App/docs/diagramas/arquitectura-auth-stego.svg	5,473 bytes
App/docs/diagramas/modelo-entidad-relacion-auth.svg	4,843 bytes
App/docs/contratos/openapi-auth.json	108,896 bytes
App/docs/contratos/openapi-stego.json	19,010 bytes
App/docs/pruebas/backend-v1.postman_collection.json	205,308 bytes
App/docs/pruebas/muestras/manifest.json	3,047 bytes
App/docs/pruebas/muestras/oculta_baja_carga.png	2,950 bytes
App/docs/pruebas/muestras/oculta_lsb.png	42,662 bytes
App/docs/pruebas/muestras/original_control.png	2,755 bytes
App/docs/pruebas/muestras/pequena.png	78 bytes
App/docs/pruebas/muestras/ruido_sin_mensaje.png	197,035 bytes
App/docs/pruebas/resultados/auth-auditoria-ejemplo.json	2,955 bytes
App/docs/pruebas/resultados/auth-funcionales-y-seguridad.txt	2,537 bytes
App/docs/pruebas/resultados/auth-unitarias.txt	400 bytes
App/docs/pruebas/resultados/integracion-auth-stego.json	1,762 bytes
App/docs/pruebas/resultados/postman-coleccion.txt	7,202 bytes
App/docs/pruebas/resultados/stego-analisis-muestras.json	5,043 bytes
App/docs/pruebas/resultados/stego-auditoria.json	13,974 bytes
App/docs/pruebas/resultados/stego-unitarias.txt	99 bytes

Los vínculos a código de los Markdown se presentan como nombres de referencia. El código ejecutable, las configuraciones de ejemplo, las migraciones y Docker Compose se entregan en el repositorio; este PDF consolida la documentación y conserva los anexos documentales.

Referencias técnicas

[1] **Consigna de la asignatura.** Proyecto Integrador del Primer Parcial. Backend: autenticación JWT/RBAC y detección de esteganografía. Fecha de entrega: 04/11/2026.

[2] **ISO/IEC 27001:2022.** https://www.iso.org/standard/27001

[3] **ISO/IEC 27034-1:2011.** https://www.iso.org/standard/44378.html

[4] **OWASP ASVS 5.0.0.** https://asvs.owasp.org/

[5] **OWASP File Upload Cheat Sheet.** https://cheatsheetseries.owasp.org/cheatsheets/File_Upload_Cheat_Sheet.html

[6] **RFC 7519 - JSON Web Token.** https://www.rfc-editor.org/rfc/rfc7519

[7] **RFC 7517 - JSON Web Key.** https://www.rfc-editor.org/rfc/rfc7517

[8] **RFC 8725 - JWT Best Current Practices.** https://www.rfc-editor.org/rfc/rfc8725

[9] **NIST SP 800-207 - Zero Trust Architecture.** https://doi.org/10.6028/NIST.SP.800-207

[10] **Westfeld y Pfitzmann - Attacks on Steganographic Systems.** https://users.ece.cmu.edu/~adrian/487-s06/westfeld-pfitzmann-ihw99.pdf

[11] **SciPy - distribución chi-cuadrado.** https://docs.scipy.org/doc/scipy/reference/generated/scipy.stats.chi2.html

[12] **Pillow - Security.** https://pillow.readthedocs.io/en/stable/handbook/security.html

Las referencias se reproducen de la consigna y de la documentación del repositorio. No se realizó una verificación bibliográfica externa al preparar este informe.